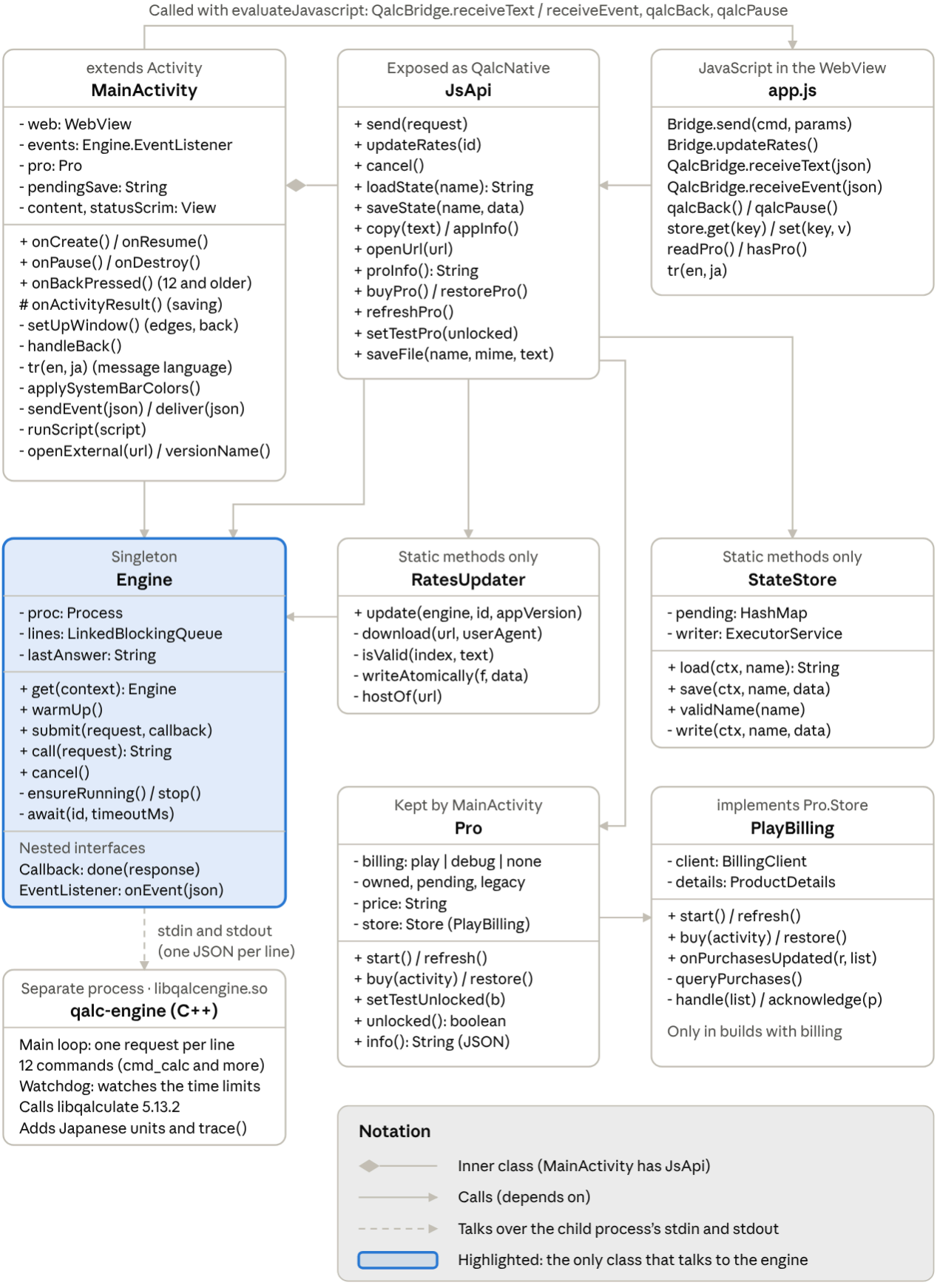
3. Detailed design

This document describes the classes, the protocol, the processing steps, the data formats and the time limits as the source code of version 1.3.0 implements them. The page's texts exist in English (the default) and Japanese; they are quoted here as the English screen shows them. The engine and the Java code return English texts, which the Japanese screen translates (3.14).

3.1 Classes and their responsibilities

JsApi takes the calls from the page; only Engine talks to the calculation engine

The 7 Java classes (package io.github.hiroshitsu.qcalc) and their links to the page and the engine



Class diagram · the 7 Java classes, the page and the engine

The Java side has seven classes (package io.github.hiroshitsu.qcalc; PlayBilling is included only when the app is built with the Google Play Billing Library, 3.20). Every call from the page goes to JsApi, and only Engine talks to the calculation engine.

Class	File (lines)	Responsibilities	Thread
MainActivity	MainActivity.java (574)	Creates and sets up the WebView (JavaScript and DOM storage on, file and content access off, text zoom 100 %). Hands replies and events to the page (evaluateJavascript). Prepares the window (setUpWindow: edge to edge on Android 15 and later, the back callback on Android 13 and later; 3.15), back (handleBack), pausing (saves with qalcPause), cleanup on exit, recreating the page when its renderer stops, external links (http and https only), system bar colors, the app version (versionName, used by appInfo and for exchange-rate downloads). Sets up QCalc Pro (creates Pro at the very start and asks for the purchases, asks again whenever the app comes back to the screen, closes the connection on exit; 3.20), saves files (opens the system file picker and writes to the chosen place on the qalc-save thread; 3.23), and picks the language of the texts Java shows itself (tr; 3.21)	UI thread

Class	File (lines)	Responsibilities	Thread
JsApi	Inner class in MainActivity.java	The 14 methods the page sees as QalcNative (3.2). Work that takes time is handed to Engine, StateStore or another thread, and the call returns at once	WebView JavaBridge thread
Engine	Engine.java (357)	Starts, watches and restarts the calculation engine. Sends a request and waits for the reply with the same id. Decides timeout, cancellation and crash. Saves ans and gives it back to a new engine	One worker thread (one request at a time), a stdout reader thread, a thread that drains stderr
StateStore	StateStore.java (123)	Saves the page's state (history, settings, the expression being typed and more) to files at once (3.12)	One writer thread
RatesUpdater	RatesUpdater.java (164)	Downloads and checks exchange rates, replaces the files and tells the engine to load them again (3.11). The User-Agent sent to the sources contains the app version passed by MainActivity	A qalc-rates thread per call

Class	File (lines)	Responsibilities	Thread
Pro	Pro.java (213)	Keeps the state of QCalc Pro (bought, payment pending, earlier user, price) and reports it to the page (info, pro event). Decides the billing kind (play, debug, none) and loads PlayBilling by its name (the app also runs without it). Saves the state in the SharedPreferences "pro" (3.20)	UI and JavaBridge threads (the state is synchronized; PlayBilling's results come on any thread)
PlayBilling	PlayBilling.java (296, app/src-billing)	Buying with Google Play Billing Library 8: connecting to Google Play, asking for the price and the purchases, the purchase screen, acknowledging purchases, messages (in English). Only in builds made with Gradle and qcalc.billing=true (3.20)	Billing Library threads (the purchase screen opens on the UI thread)

Main calls

- MainActivity → Engine: warmUp at start (starts the engine early), registers and removes the event listener.
- JsApi → Engine: send calls submit (handled one at a time on the worker thread); cancel stops the engine.
- JsApi → RatesUpdater: updateRates calls update on a new thread.
- RatesUpdater → Engine: call sends rates_info (id −2) and rates_reload (id −3).
- Engine → qalc-engine: one request line on standard input; replies and events, one line each, on standard output.
- MainActivity → Pro: creates Pro first thing at start and calls start (connects to Google Play and asks for the purchases), refresh whenever the app comes back to the screen (onResume), end on exit (onDestroy). Pro → PlayBilling: the Store interface (start, refresh, buy, restore, end). PlayBilling → Pro: the Owner interface (owned, price, message). Pro → MainActivity: the Listener (pro events for the page).

- JsApi → Pro: proInfo, buyPro, restorePro, refreshPro, setTestPro. JsApi → MainActivity: saveFile opens the file picker; onActivityResult writes the file on the qalc-save thread and sends a saved event.

The page (app.js) is described in 3.7, and the calculation engine (qalc-engine.cc) in 3.3 to 3.6.

3.2 The JavaScript bridge

From the page to Java: the 14 methods of window.QalcNative. From Java to the page: the 2 methods of window.QalcBridge and 2 hooks of the page. The reply to a calculation is not a return value: it comes later, matched by its request number (id).

Page → Java (window.QalcNative, run on the JavaBridge thread)

Method	Arguments	Returns	What it does
send(request)	A one-line JSON request	Nothing	Handled on Engine's worker thread; the reply comes back through QalcBridge.receiveText
updateRates(id)	Request number (integer)	Nothing	Updates the exchange rates on a separate qalc-rates thread; the result comes through receiveText
cancel()	None	Nothing	Stops the engine process that is calculating
loadState(name)	Saved name	String or null	Reads state/name.json; a newer value not yet written is returned instead
saveState(name, data)	Saved name, JSON string (null deletes)	Nothing	Saved later on the writer thread
copy(text)	String	Nothing	Puts the text on the clipboard (label "QCalc"). Android 12 and older show "Copied" (13 and later show their own notice). When the clipboard does not take it (Android takes about 1 MB at a time), the exception is caught and "Too long to copy" is shown (1.2.0)

Method	Arguments	Returns	What it does
appInfo()	None	JSON string {version, android, device, dark}	The app version, the Android version, the device name, whether dark mode is on
openUrl(url)	URL	Nothing	Opens http and https URLs in the external browser
proInfo()	None	JSON string {billing, pro, legacy, price, state}	The state of QCalc Pro (3.20). billing: play, debug or none; pro: Pro can be used; legacy: used as an earlier user; price: Google Play's price text (¥300 and the like); state: pending while a payment is processed
buyPro()	None	Nothing	Opens Google Play's purchase screen on the UI thread (only with PlayBilling). The result comes as a pro event
restorePro()	None	Nothing	Asks for the purchases again. When there is none, the pro event's message says No purchase of QCalc Pro was found
refreshPro()	None	Nothing	Asks for the price and the purchases again (when the QCalc Pro panel opens)
setTestPro(unlocked)	Boolean	Nothing	Test builds only (billing debug): turns Pro on or off, saves it and sends a pro event. Does nothing in other builds

Method	Arguments	Returns	What it does
saveFile(name, mime, text)	File name, type, contents	Nothing	Lets the user choose where to save with the system file picker (ACTION_CREATE_DOCUMENT) and writes the contents in UTF-8. The name: up to 100 letters, digits, ".", "_" and "-", starting with a letter or digit; the type: text/csv only; the contents: up to 16,777,216 characters (16 × 1024 × 1024). The result comes as a saved event (3.23)

Java → page (evaluateJavascript on the UI thread)

Call	Contents
QalcBridge.receiveText(text)	One reply line. Parsed, and the Promise waiting for the same id is resolved. If it cannot be parsed, the oldest waiting request fails
QalcBridge.receiveEvent(text)	One event (JSON) of three kinds: ans (the engine's ans event: attaches the text to insert to the history item), pro (the state of QCalc Pro changed: PRO is read again and a message, if any, is shown; 3.20), saved (the result of saving a file: ok, cancelled or error; 3.23)
window.qalcBack()	Back. true when the page handled it (Java waits 0.8 seconds; without an answer the app goes to the background)
window.qalcPause()	The app goes to the background. The history and the expression being typed are saved at once

Every wait in the page has a last-resort timer: for calculations the larger of “2 × time limit × number of expressions + 60 seconds” and 120 seconds, for exchange rates 300 seconds. After it, the request ends with “No reply from the calculation engine”. Normally the Java limit (time limit + 10 seconds) acts first.

3.3 Protocol between the app and the engine

The app writes one JSON request per line to the engine's standard input, and the engine writes one JSON reply per line to its standard output. Requests are handled one at a time in order, and each reply carries the id of its request.

Common form

- Request: `{"id": number or string, "cmd": "command", …}`. The page numbers its requests from 1; Java uses −1 (giving back ans) and −2 and −3 (exchange rates) itself.
- Success: `{"id":…, "ok":true, …}`. Failure: `{"id":…, "ok":false, "error":"message"}`. A request that is not valid JSON, or has extra characters after it, gets `{"id":null, "ok":false, "error":"bad request"}`.
- Error texts and messages are in English (the texts of the engine and the Java code since 1.3.0; before, they were in Japanese). On the Japanese screen the page translates them with MESSAGES_JA (3.14, 3.21).
- When the engine is going to exit after a reply, the reply carries `"restart":true`. Java throws the process away and starts a new one with the next request.
- When a commit (store:true) changes ans, an event line follows the reply at once: `{"event":"ans", "id": id of that calculation, "value":"expression that recreates ans", "input":"text to insert"}`. Java does not take this line as a reply: it saves value and forwards the event to the page.
- Text that libqalculate itself writes to standard output is moved to standard error at start, so that it does not mix with the replies. Java reads standard error and drops it.

Commands

Command	Request fields	Reply fields	Main errors
init	None	engine ("libqalculate 5.13.2"), rates_time (time of the exchange rates, UNIX seconds), ans_defined (whether ans has a value), defs_error (only when a definitions file was moved aside)	—

Command	Request fields	Reply fields	Main errors
calc	expr, opts (3.4), timeout (ms, 100 to 120000, default 3000), store (default false), preview (default: the opposite of store), max_chars (the most bytes of the result, 100 to 10000000, default 200000), tail_chars (bytes of the end also returned when the result is longer than max_chars: 0 to 1000000, default 0, used only when twice it is at most max_chars; 1.3.0)	result, parsed, approx, comparison, timeout, truncated, chars and digits (only when cut: characters and digits of the whole), tail (only when cut with tail_chars: the end), messages (type error, warning or info), deferred (when a preview did not run the expression)	The expression is nested too deeply
set_ans	expr	set (whether it was set)	expr missing
plot	exprs (array of strings, at most 12), min and max (default "-10" and "10"), steps (10 to 4000, default 300), var (default "x"), opts, timeout (default 8000, shared by all expressions)	series (per expression {x:[…], y:[…]}, null where there is no value), messages	exprs missing, too many expressions, invalid range
plot3d	exprs (array of strings, at most 4), xmin, xmax, ymin, ymax (default "-5" and "5"), nx and ny (grid points, 2 to 121, default 41), opts, timeout (default 15000, shared by all expressions)	x (nx values), y (ny values), series (per expression z: ny × nx values row by row, null where there is no value; complete: whether every row was calculated), messages	exprs missing, too many expressions, invalid x range, invalid y range

Command	Request fields	Reply fields	Main errors
catalog	kind (functions, variables, units)	items (functions: name, title, category, description, syntax, local; variables: name, display, title, category, description, local, value; units: name, abbr, display, title, category, description, currency, local, composite, insert). A unit's abbr is the text shown in the list and inserted into expressions (km, km/h, kW·h and the like); the engine checks that it reads back as the same unit before returning it (3.18). composite marks compound units (km/h and the like); insert=false marks units that cannot be inserted into an expression and can only be a conversion target (after to)	unknown catalog kind
rates_info	None	sources (index, url, file, time), time	—
rates_reload	None	loaded (whether the files were loaded), time, messages	—
define_var	name, value, title (optional)	saved (whether it was written to the file), messages	Enter a name and a value, The variable name contains characters that cannot be used, "name" is a built-in name and cannot be used, Check the value (…)

Command	Request fields	Reply fields	Main errors
define_func	name, expr (arguments x, y, z), title (optional)	saved, messages	The same (except the variable-name text), and The function name contains characters that cannot be used, "to" cannot be used in the expression of a function
delete_def	kind (variable, function), name	saved	There is no user definition "name"
user_defs	None	variables (name, value, title, temporary), functions (name, expr, title)	—

Example (a commit: the request, its reply and the ans event)

```
{"id":19,"cmd":"calc","expr":"7/3","opts":
{"approx":"auto","precision":10},"timeout":20000,"store":true,"max_chars":450
000,"tail_chars":20000}
{"id":19,"ok":true,"result":"2 + 1/3 ≈ 2.333333333","parsed":"7 /
3","approx":false,"comparison":false,"timeout":false,"truncated":false,"messa
ges":[]}
{"event":"ans","id":19,"value":"7/3","input":"7/3"}
```

Example (a 3D graph: the request and its reply for a 3 × 3 grid; the second expression is not real for x < 0)

```
{"id":7,"cmd":"plot3d","exprs":
["x*y","sqrt(x)"],"xmin":"-1","xmax":"1","ymin":"-1","ymax":"1","nx":3,"ny":3
}
{"id":7,"ok":true,"x":[-1,0,1],"y":[-1,0,1],"series":[{"z":
[1,0,-1,0,0,0,-1,0,1],"complete":true},{"z":
[null,0,1,null,0,1,null,0,1],"complete":true}],"messages":[]}
```

3.4 Calculation options

The page sends its settings as opts with every request, and the engine builds the options anew from libqalculate's defaults every time (make_opts). Because libqalculate leaves some fields of its structures uninitialized, the engine always copies static defaults first

and then changes them.

Settings that never change (as in the official qalc)

- Input: comma is a separator, dot the decimal point. Implicit multiplication is not limited. Adaptive parsing. Unknown names are errors, not symbols (unknowns_enabled = false).
- Calculation: a warning when it is assumed that a denominator is not zero. Currencies are converted to the local currency (JPY in Japan).
- Display: Unicode signs (×, −), "/" for division, logic as true, false, or and and. Number bases with 0x, 0b and 0o, and a capital E for exponents. Uncertainty as ± when the input has ±, as an interval with interval(), otherwise by significant digits.

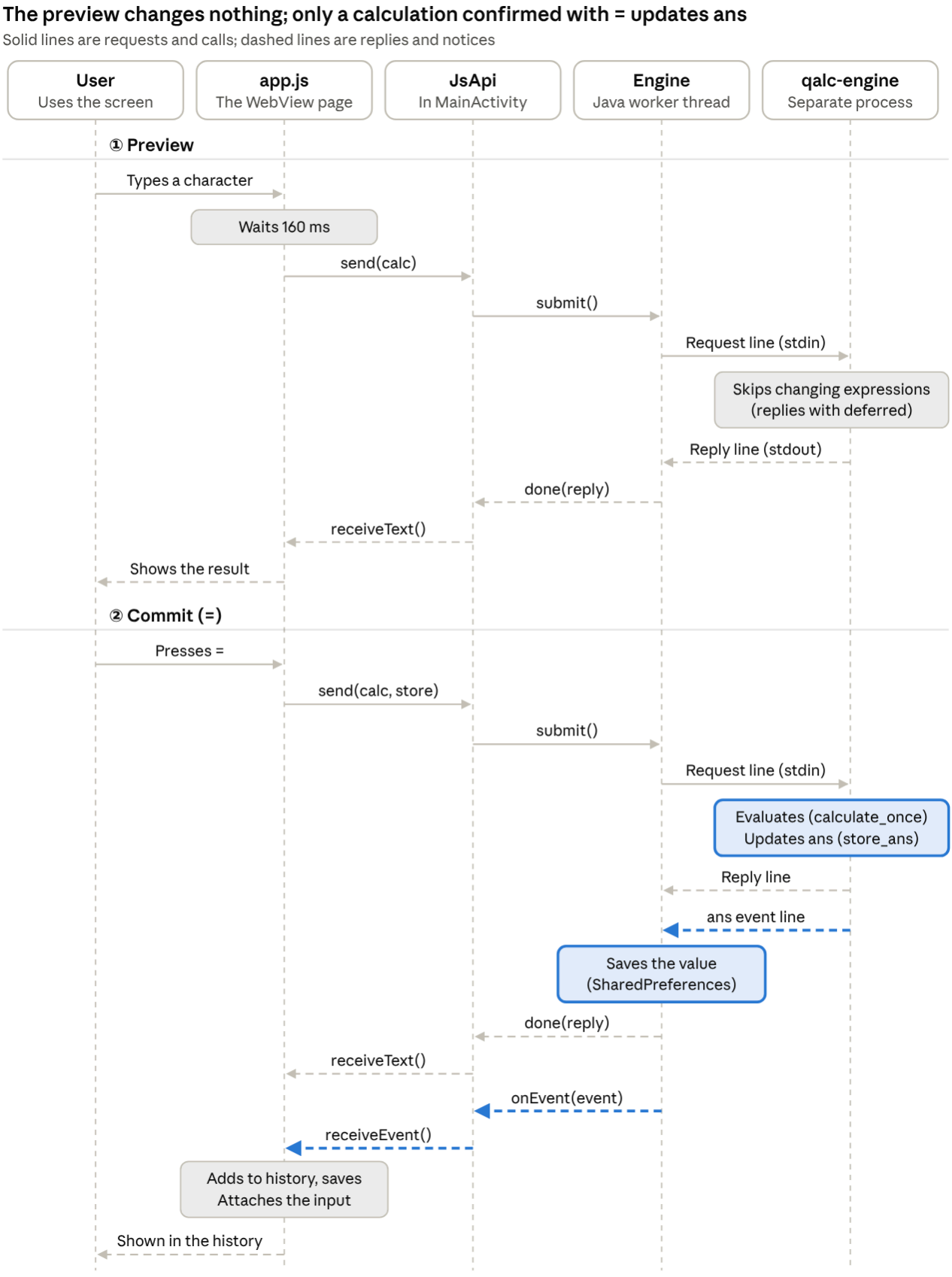
The page's settings

opts field	Values	libqalculate setting	Setting on the screen
approx	auto (default), exact, approx	APPROXIMATION_TRY_EXACT + automatic approximation AUTO, APPROXIMATION_EXACT, APPROXIMATION_APPROXIMATE	Exact or approximate (also the button at the top)
fraction	auto (default), decimal, exact, fraction, combined	Automatic fractions AUTO, FRACTION_DECIMAL, FRACTION_DECIMAL_EXACT, FRACTION_FRACTIONAL, FRACTION_COMBINED (the last two limit the length of fractions)	Fractions
angle	rad (default), deg, gra	ANGLE_UNIT_RADIANS, DEGREES, GRADIANS	Angle unit (also the button at the top)
base	10 (default), 16, 2, 8, 12, -1	Number base of results (-1 is Roman numerals; 2 to 36 accepted)	Number base of results (the top button: 10, 16, 2, 8)
precision	2 to 200000 (default 10; 1000 up to 1.1.0. Without QCalc Pro the page sends at most 1000; 3.20)	setPrecision (significant digits)	Precision (significant digits)
max_decimals	-1 (default, no limit), 0, 2, 4, 6, 8, 10	use_max_decimals and max_decimals	Maximum decimal places

opts field	Values	libqalculate setting	Setting on the screen
notation	auto (default), sci, eng, pure, none	EXP_PRECISION, EXP_SCIENTIFIC, EXP_BASE_3, EXP_PURE, EXP_NONE	Exponential notation
units	optimal (default), si, base, none	POST_CONVERSION_OPTIMAL, OPTIMAL_SI, BASE, NONE	Automatic unit conversion
mixed_units	true (default), false	MIXED_UNITS_CONVERSION_NONE when false	Mixed units
complex	rect (default), exp, polar, cis	COMPLEX_NUMBER_FORM_RECTANGULAR, EXPONENTIAL, POLAR, CIS	Complex number form
structuring	simplify (default), factorize, expand	STRUCTURING_SIMPLIFY, FACTORIZE, EXPAND	Simplification
grouping	false (default), true	DIGIT_GROUPING_STANDARD when true (the engine groups with narrow spaces, in the decimals too; the page shows only the groups of the integer part with commas; 3.9)	Digit grouping

The engine also accepts dual for approx and fraction, and an input base input_base (2 to 36), but the screen has no setting for them.

3.5 Calculation



Sequence diagram · the live preview and the commit (=)

The preview and the commit are both calc commands. They differ only in store (whether ans is updated), preview (whether expressions that change something are left unrun), the time limit and the length of the result.

calc (cmd_calc)

1. Reads timeout (100 to 120000 ms, default 3000), max_chars, tail_chars, store (default false) and preview (default: the opposite of store), and builds the calculation options from opts (3.4).
2. Refuses an expression nested too deeply with The expression is nested too deeply.
3. In a preview, an expression is not calculated, and deferred=true is returned, when it assigns (x = 5, a := 1 and the like) or contains save(), or uses the functions save, export, command, register, stack or plot, or a user function that contains save, export or :=. This keeps definitions and saving from running again and again while an expression is being typed.
4. Evaluates once with calculate_once (next).
5. With store, when there is a value and no timeout, updates ans with store_ans.
6. When the calculation thread had to be cancelled by force, adds restart=true to the reply and exits after the reply (a new engine starts with the next request).
7. When result is longer than max_chars bytes, counts the characters (chars) and the digits (digits) of the whole. If tail_chars is given and twice it is at most max_chars, takes the last tail_chars bytes as tail and cuts the beginning at max_chars − tail_chars bytes (without tail, at max_chars bytes). The cut beginning ends with "…", and nothing is cut in the middle of a character. parsed is cut at 2,000 bytes. Then the reply is written; when ans changed, the ans event follows.

Evaluating once (calculate_once)

The same steps as libqalculate's calculateAndPrint, with the expression evaluated only once (so that random numbers and expressions such as a = a + 1 are not evaluated twice). The value evaluated becomes ans, so the value shown and ans always agree.

1. Removes comments (after #). An expression that is only a comment does nothing.
2. Parses the conversion target after "to" separately (bases, fractions, factorization, partial fractions, calendars, units, binary prefixes and more).
3. Removes factor, factorize or expand at the beginning and uses that kind of simplification.
4. Turns an assignment such as "x = 5" into saving a variable, with the same rules as qalc (transform_expression_for_equals_save).
5. Evaluates on libqalculate's calculation thread with the time limit. When it does not finish in time, timeout=true; when the thread is cancelled by force, as in step 6 above.

6. Formats the result within the time left (at least 200 ms). A result that keeps its units as they are is converted to the best unit. When both an approximate and an exact value mean something and less than 1/3 of the time limit was used, the exact value is also found and shown beside it, as in “7/3 ≈ 2.333333333” (print_dual). In automatic mode, though, when the exact form contains a sum of 6 terms or more, or has more than 30 elements in all, only the approximate value is shown (the limit 5 of calculate_dual_exact, as in qalc; matched in 1.1.0).
7. A value with uncertainty (± or uncertainty()) is shown with ±, interval() as an interval, and other values by significant digits.
8. Finds approx (the result is approximate), comparison (the result of a comparison or an equation) and parsed (the expression as parsed). On a timeout, result is empty.

Collecting messages (collect_messages)

The warnings and errors of a calculation are taken from libqalculate's message list just before the reply is made, put in messages, and the list is cleared. Before that, any “message stop” still open is closed, and the messages held in it are added to the list too. This works around libqalculate 5.13.2 leaving such a stop open (3.19).

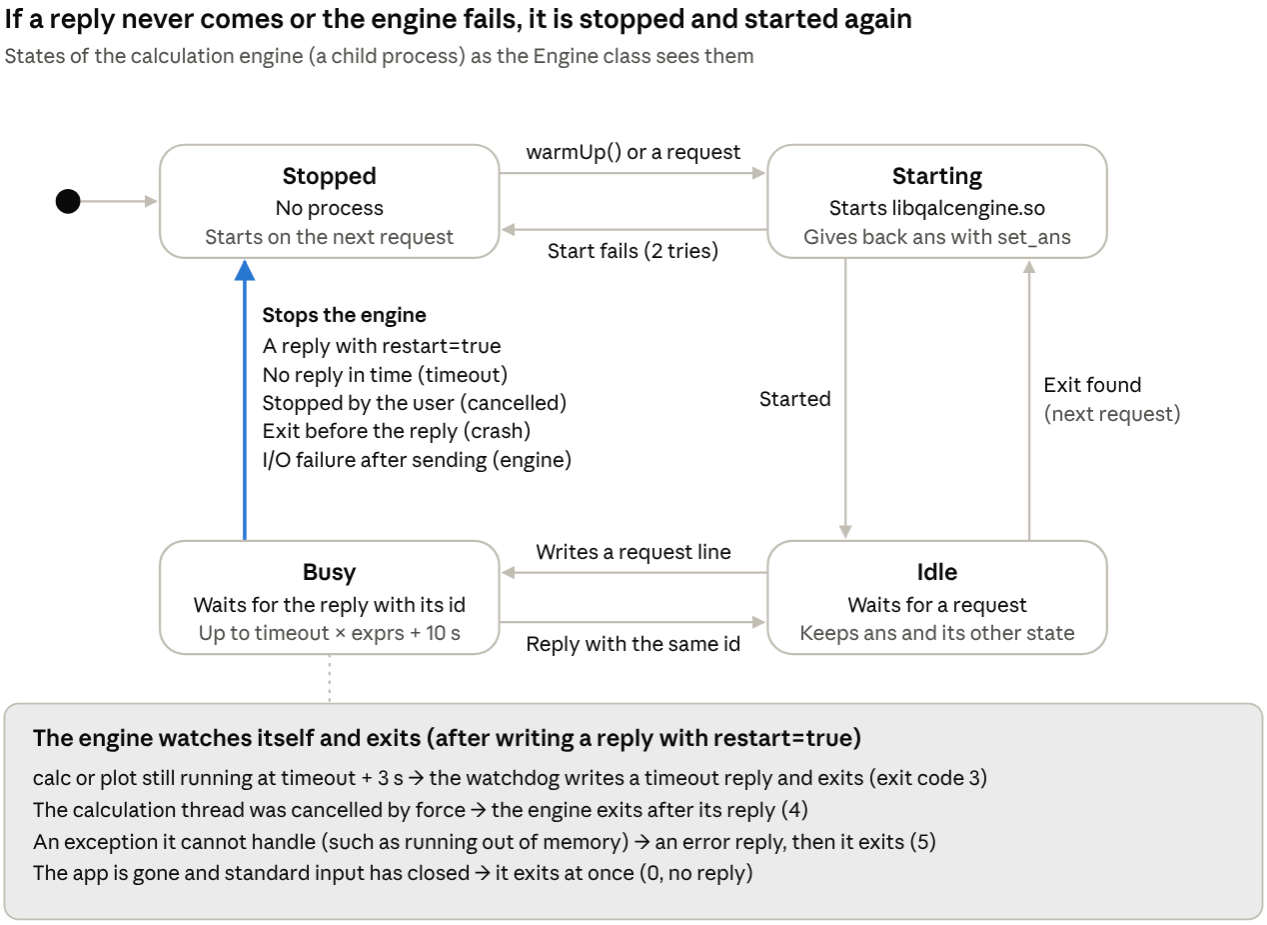
Rules for ans (store_ans)

- ans changes only on a commit (store=true) with a value and no timeout. A preview leaves it unchanged (but can use ans).
- Before the new value goes in, ans5 ← ans4 ← ans3 ← ans2 ← ans are shifted (as in qalc).
- For an equation, the solution goes in: x = 2 gives 2, x = 2 or x = −2 gives [2 −2]. When a solution has a condition joined with and, the right side of the equation is used. Otherwise the result itself.
- After an update, the ans event follows the reply. value is libqalculate's saved form (to give ans back to a new engine); input is a text that can be typed back into an expression (decimal, or a fraction when a decimal would not be exact; no digit grouping; all digits of exact values).
- Java saves value (up to 500,000 characters) and gives it back with set_ans (id −1) whenever it starts a new engine. set_ans reads the value with the precision raised for a while to the length of its longest run of digits + 10 (at most 200,010; a new engine starts with precision 10, and without raising it an approximate value would be rounded to about 38 digits). A number of 2,000 digits or more is not passed to libqalculate but read directly with GMP, put into the expression as a temporary variable (qcalc_long_number_0 and so on) and deleted after the calculation. libqalculate reads numbers one digit at a time, so its time grows with the square of the length (about 3.4

seconds for a 200,000-digit value), while GMP takes about 0.2 seconds with the same result (H15 in the test specification, 6.2). The time limit is 15 seconds. Only ans is given back, not ans2 to ans5.

- When the engine reports ans_defined=false at start and there is a history, the page sends set_ans with the insert text of the last history item (carried over from earlier versions).

3.6 Engine process states and supervision



State diagram · the four states of the calculation engine and recovery

Engine runs exactly one calculation engine and handles requests one at a time (one worker thread, and call is synchronized). Only cancel stops the process without taking the lock, because the waiting call holds it.

Waiting for a reply (await)

- The stdout reader thread puts each line in a queue. An ans event line is not queued: it is saved on the spot and passed to the page. When the engine exits, an end marker (EOF) is queued.

- The waiting side takes as the reply only a line that starts with `{"id":` and the same id followed by `,` and ends with `}`, and skips other lines (a late reply to an earlier request, a cut line and so on).
- It waits timeout × number of expressions + 10 seconds (30 seconds when there is no timeout).

Recovery rules

Situation	What Java does	Reply to the page
The reply has restart=true	Passes the reply on and stops the process (a new one starts with the next request)	The engine's reply
The wait ran out	Stops the process	code=timeout
The user cancelled	cancel stops the process, and the waiting request ends	code=cancelled
The engine exited after the request was sent (EOF)	Stops it. The same request is not sent again (the same expression would likely end it again)	code=crash
Reading or writing failed after sending	Stops it	code=engine
Writing failed before sending (the engine had already exited)	Starts it again and sends once more	code=engine if the second try fails too
The engine had exited while idle	Notices the exit at the next request, starts it again and then sends	As usual

A restarted engine gets the saved ans back with set_ans (id −1, waiting up to 20 seconds). User definitions are read from their files again, so a restart does not change them for the user; ans2 to ans5 and temporary variables (x = 5 and the like) are lost.

Supervision inside the engine (qalc-engine.cc)

- When calc, plot or plot3d starts, the deadline (timeout + 3 seconds) and the request id are handed to the watchdog thread. Every 0.2 seconds it checks the deadline; once it has passed, the watchdog writes a reply with timeout=true and restart=true and exits (exit code 3). Writing a reply clears the deadline.

- libqalculate's own abort waits about 5 seconds plus the cube root of the precision in seconds before it cancels the calculation thread. Replying earlier and recreating the engine gets the user back sooner.
- After the calculation thread has been cancelled by force, libqalculate's state is not safe, so the reply carries restart=true and the engine exits (4).
- When an exception (such as std::bad_alloc with a huge result) happens while a request is handled, the engine replies "engine error: …" with restart=true and exits (4). For an exception that cannot be caught (on the calculation thread and the like), on_terminate writes Not enough memory for the calculation with restart=true and exits (5).
- So that replies and logs do not mix, standard output is duplicated at start for replies only, and the original standard output (which libqalculate sometimes writes to) is connected to standard error. Java drops standard error.
- Every reply line is written under a lock, so a watchdog reply and a normal reply never mix.
- When standard input closes (the app process is gone), the engine exits at once without waiting for the calculation thread (0).

Starting the engine (main)

1. Creates libqalculate's Calculator and the place for its settings and definitions (under the environment variable QALCULATE_USER_DIR).
2. Loads the exchange rates (loadExchangeRates) and the built-in definitions (loadGlobalDefinitions).
3. Adds 10 Japanese units (add_japanese_units) and trace() (add_matrix_functions; 3.19).
4. Checks the user definition files (moves aside the ones that cannot be read; 3.12) and loads them (loadLocalDefinitions). Messages up to here are dropped.
5. If the environment variable QALC_CURRENCY (the currency Java chose from the phone's region; JPY in Japan) names a currency, makes it the local currency.
6. Creates the variables ans and ans2 to ans5 (undefined at first).
7. Duplicates standard output for replies only, connects the original standard output to standard error, starts the watchdog thread and waits for requests.

For every request, before the command runs, any message stop still open is closed (nothing is carried over from the previous command; what it held is dropped; 3.19).

3.7 The page (app.js) and its sections

app.js is one immediately invoked function holding 23 sections in a row (2,653 lines), with no outside libraries. The line numbers are those of 1.3.0. Every text on the screen is chosen from an English and Japanese pair with tr(English, Japanese) (3.21). The license texts are in a separate file, licenses.js, loaded only when needed (3.16).

Lines	Section	Main functions and data	Role
7–84	Common	el, $, store, LANG, JA, tr, debounce, vibrate, own, toast, hideToast, copyText, openUrl	Making elements, saving (to files and to localStorage), the language and choosing texts (3.21), short notices (also with a button such as Undo)
85–143	Bridge	Bridge.send, updateRates, backstop, receive, receiveText, receiveEvent	Request numbers, the waiting requests (Map), the last-resort timer, passing events on (ans, pro, saved)
144–161	Settings	DEFAULTS, TEXT_SCALE, textScale, applyTextSize, S, saveSettings	Defaults of the 19 settings, text size, loading and saving the settings
162–199	Pro	MAX_PRECISION, FREE_MAX_PRECISION, PRO, readPro, proOffered, hasPro, maxPrecision, engineOpts, labels of the top buttons	The state of QCalc Pro (3.20), the highest precision, the opts sent to the engine
200–239	History data and elements	MAX_RESULT, TAIL_RESULT and more, history, fitHistory (size of the history), saveHistory, clip (beginning of a long text), exprEl, histPinned and more	Loading and saving the history (in batches of 0.3 seconds), shortening old long results beyond the budget, keeping the newest item in view (ResizeObserver)
240–389	Keys	PAGES (4 pages × 25 keys), PAGE_TABS, TOOLBAR, KEY_NAMES, makeKey, bindPress, buildKeypad, setPage, renderPage, updateToolbar	Keys and tabs, switching tabs, presses and repeats, names for screen readers (English and Japanese)

Lines	Section	Main functions and data	Role
390–570	Editing	offerUndo, dropUndo, setExpr, insert, action, toggleKeyboard, autosize, normalizeJa, normalizeField	Inserting (templates, wrapping a selection), deleting, undoing a cleared expression, moving the cursor (selections and emoji), keyboard mode, converting full-width characters and the like, Enter
571–790	Preview and matrices	exprChanged, saveDraft, runPreview, displayText, shownText, displayResult, splitMatrices, showResult, MESSAGES_JA (57), localMessage, showPreview, cutMessage	Calculating while typing, how results are shown (digit groups, the beginning and end of long results), matrices as tables, Japanese messages
791–884	Commit	commit, setInput, receiving events (ans, pro, saved), announce	The commit, adding to the history, attaching the insert text, cancelling
885–1033	History	insertText, wrapValue, legacyInsertText, insertResult, renderHistory, fitResultBox, fitMatrices, bindLongPress, historyMenu, historyChanged, tapButton	Showing the history (the first screen when it is empty), taps and long presses, reusing results, undoing a deletion
1034–1131	Sheets and menu	sheet, closeSheet, confirmSheet, MENU (13 items), buildDrawer, menuAction	Choice sheets, confirmations, the menu (no QCalc Pro item in builds where it cannot be bought; PRO marks on Pro features), clearing the history and undoing it
1132–1161	Panels	panel, closePanel, panelStack	Making panels (once) and stacking them
1162–1297	Examples	STARTERS (4), EXAMPLES (12 topics, 69 examples), TIPS (11), useExample, openExamples	Examples and tips; a tapped example goes into the input field (3.18)

Lines	Section	Main functions and data	Role
1298–1541	Lists	loadCatalog, HIDDEN_ITEMS, CATEGORY_JA, JA_NAMES (124), FUNC_JA (230), VAR_JA (61), JA_UNIT_TEXT, openCatalog, catalogRow, useItem	Getting the lists, Japanese names and search (Japanese screen only), opening a category, inserting (3.18)
1542–1836	Graph (2D)	openPlot, compute, autoView, niceStep, gridValues, viewOk, draw, zoom, refetchIfNeeded, setupPlotGestures, showPoint	Switching between 2D and 3D (3D is QCalc Pro), getting the points, drawing on the canvas, gestures
1837–2141	3D graph	Q3, VIEW3D, viridis, build3d, zRange3d, compute3d, draw3d, pick3d, setup3dGestures, zoom3d	Getting the grid values, the z range, projection and shading, drawing from the back, turning, zooming, showing coordinates (3.10)
2142–2204	User definitions	openDefs, invalidateCatalogs	The list, adding, deleting
2205–2242	Exchange rates	openRates, fmtTime	Updating by hand
2243–2355	Settings	THEMES (7 colors), SETTINGS (19), openSettings, settingsChanged, updateChips, cycle	The settings panel (checks for Pro settings, switching the language) and the buttons at the top
2356–2424	About this app	openAbout, PRIVACY_URL, SOURCE_URL, loadLicenses, openLicenses	Version, link to the examples, privacy, the license list and texts (3.16)
2425–2498	QCalc Pro	proPrompt, PRO_FEATURES, openPro, fillPro, buyPro, restorePro, proChanged	The Pro prompt, the QCalc Pro panel (buying, restoring, the test switch), reacting when the state changes (3.20)
2499–2543	Export	historyCsv, plainNumber, exportHistory, fileSaved	Making and saving the CSV file (3.23)

Lines	Section	Main functions and data	Role
2544–2653	Back, pause, start	qalcBack, qalcPause, applyTheme, autoUpdateRates, localizeStatic, chooseLanguage, setLanguage, boot, startEngine	Back, going to the background, color themes (3.22), language (3.21), start, automatic exchange-rate updates

3.8 Input

The input field is one textarea. With the on-screen keys the phone's keyboard stays hidden (inputmode="none"); in ⌨ mode it becomes ordinary text input. Both kinds of input end in the same handling of a change (exprChanged).

Keys and tabs (PAGES, PAGE_TABS, setPage)

- PAGES holds the definitions of 25 keys for each tab id (num, fn, calc, mat). A definition is a set of: the key's text; ins, the text it inserts; back, how far to move the cursor back after inserting; op, the operator mark (ans is put in front in an empty field); act, an action (del, clear, eq, more, kbd, left, right); repeat when held; html with subscripts and superscripts ($\int_a^b$, A^T, I_n) and more. The key layout is in the basic design, 2.5.
- PAGE_TABS has, for each of the four tabs, its id, its label (123, Functions, Calculus, Matrix; on the Japanese screen 123, 関数, 解析, 行列), its name for screen readers, and the category of the function list: Calculus for Calculus, Matrices & Vectors for Matrix. The "more…" key uses it.
- buildKeypad makes the 8 keys that are always shown (TOOLBAR) and the 4 tabs (#tab-num, #tab-fn, #tab-calc, #tab-mat, role="tab"). setPage(id) fills the 5 × 5 area (#grid) with the chosen tab's keys and gives that tab the on class and aria-selected="true". The tab is not saved (the app starts on 123).
- "more…" (act more) opens the function list with the current tab's category (openCatalog('functions', category); 3.18).

Key presses (bindPress)

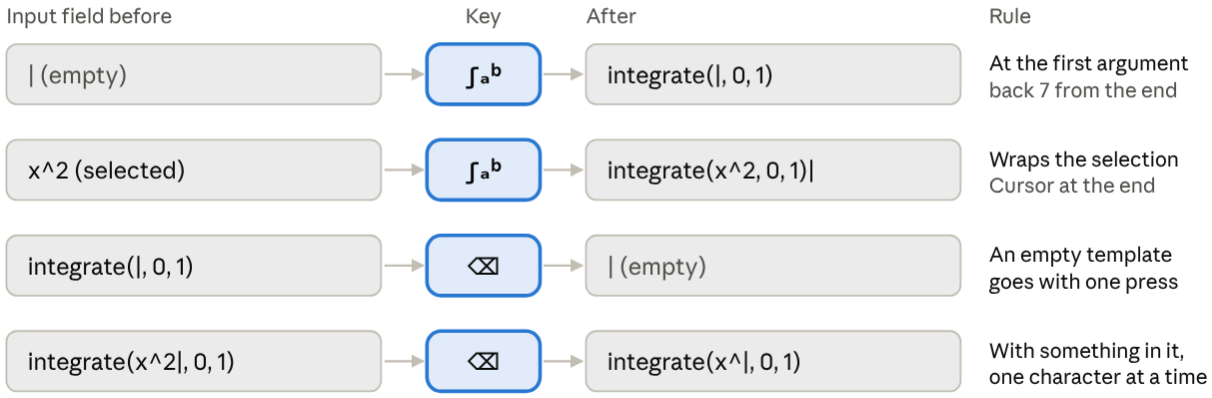
- pointerdown cancels the default action, so that the input field keeps its focus and cursor; the key acts on pointerup.
- ⌫, ◀ and ▶ repeat when held: after 420 ms, every 70 ms. They stop when the finger leaves the key or the touch is cancelled.
- An ordinary click (TalkBack, a keyboard) also works. A click that follows a touch is ignored, so that nothing is typed twice.

Inserting and deleting (insert, action)

- Text goes in at the cursor. A key's back says where the cursor goes, counted back from the end of the inserted text: 1 for sin() (inside the parentheses), 7 for integrate(, 0, 1) (the first argument). With a selection and a back of 1 or more, the selected text is put at that place, wrapped (select x^2 and press $\int_a^b$: integrate(x^2, 0, 1)), and the cursor goes to the end of the inserted text.
- When the field is empty, there is a history and the setting "An operator in an empty field continues from ans" is on, an operator key puts ans in front. The same happens when a single *, /, ^, ×, ÷ or · (full-width too) is typed on the phone's keyboard.
- ⌫ deletes the selection if there is one. When the text before the cursor is "name(" or √(, ∛(, ∜(, and what follows is ")" or only arguments without parentheses or separators ending in ")" (such as ", 0, 1)"), the empty function or template goes at once (sin(|) and integrate(|, 0, 1) go with one press; dsolve(diff(y, x) = |) one character at a time). Emoji and other characters of two units go at once.
- ◀ and ▶ move the cursor to the start (◀) or the end (▶) of a selection and drop the selection. Without one they move by one character, over emoji and other surrogate pairs at once.

A template goes in with the cursor where the expression is written

The $\int_a^b$ key of Calculus (it types integrate(, 0, 1)) and ⌫; | marks the cursor



Inserting and deleting a template · the $\int_a^b$ key and ⌫ in four cases

Undoing a cleared expression (offerUndo, dropUndo, setExpr)

- The expression cleared with AC, or replaced by tapping an expression in the history, by "Edit expression" or by tapping an example (setExpr), is remembered (only one: undoText), and the "↶ Undo" button (#undo) appears at the top right of the input field. An expression of only spaces is not remembered.
- The button puts the remembered expression back, with the cursor at its end.
- Inserting with a key, deleting with ⌫, typing on the phone's keyboard or a commit drops the remembered expression and hides the button (so that new input is never overwritten).

- Undoing “Delete this line” and “Clear history” works differently, with the button of a notice (3.9).

Committing from the keyboard

- Enter on keydown (not while Japanese text is being converted) and a line-break beforeinput (insertLineBreak, insertParagraph) commit.
- If a line break still gets in, it becomes a space, and a line break at the end commits.

Converting characters (normalizeJa)

Every replacement turns one character into one character, so the cursor position stays valid. Nothing is converted while Japanese text is being converted; it happens when the conversion ends (compositionend). The same conversion is applied before sending in the preview, the commit, graphs and user definitions, on the English screen too.

Input	Becomes	Example
Full-width letters, digits and signs (U+FF01 to U+FF5E)	Half-width	１０＋５ → 10+5
Full-width space (U+3000)	Half-width space	—
円, ￥	¥ (read as Japanese yen)	100円 → 100¥
Hyphens (U+2010, U+2011, U+2013 to U+2015)	-	—
ー between digits, parentheses and spaces	-	５ー３ → 5-3 (names such as コーヒー stay)

Keyboard mode (toggleKeyboard)

- ⌨ switches inputmode between text and none. With text, the body gets the kbd class, which hides the 5 × 5 keys and the tabs.
- When the screen grows taller than 1.3 times the smallest height since the mode began (the phone's keyboard closed), the on-screen keys come back. When the width changes (rotation), the base is measured again. When the height shrinks, the history scrolls to the newest item.

Preview (exprChanged, runPreview)

- After every change, the expression being typed is saved (in batches of 0.5 seconds) and the input field's height is adjusted (at most 110 px × the text size factor).

- If the field is empty or the preview is off, the preview is cleared. Otherwise it is calculated after 160 ms (timeout 1500, max_chars 450000, tail_chars 20000: the same lengths as the commit since 1.3.0; before, max_chars 6000).
- Only one preview calculation runs at a time. Changes meanwhile are calculated once when it ends. The result is shown only if the input field has not changed and no commit is running.
- For an expression the engine did not run (deferred), “Press = to run it (definitions are not previewed)” is shown.

Text size (applyTextSize)

The setting text_size (normal, large, xlarge) is written to the data-text-size attribute of the html element, and the CSS variable --fs (1, 1.18, 1.38) sets the size of the text in the history (expressions, results, messages), the parsed expression, the input field, the preview and the messages. Keys and panels keep their size. The input field’s maximum height (110 px) grows by the same factor. It is applied at start (boot) and when the settings change (settingsChanged).

3.9 History and reusing results

The commit (commit)

1. Nothing happens if the field is empty or a commit is running. If the engine has not finished starting (init), “The calculation engine is starting” is shown.
2. The expression is converted with normalizeJa, the planned preview is cancelled and “Calculating…” is shown. If it has not finished after 1.2 seconds, it becomes “Calculating… (tap here to stop)”. A tap calls QalcNative.cancel() and shows “Stopping…”.
3. calc is sent with store=true, timeout 20000, max_chars 450000 (MAX_RESULT: room for a complex number with 200,000-digit parts, or 200,000 digits with digit grouping) and tail_chars 20000 (TAIL_RESULT).
4. On failure (ok=false), “The calculation was stopped” (code=cancelled) or the error text; on a timeout, “The calculation took too long and was stopped”; with an empty result, the first error message or “Check the expression”. These are shown in red, and nothing is added to the history.
5. On success, a history item is made and added at the end (at most 300; the oldest go first) and saved in a batch of 300 ms.
6. If the input field still holds what was sent, it is emptied. If something was typed during the calculation, it stays and the preview runs again.
7. The history is drawn again and scrolled to the newest item, and “expression, result” is put in #status for screen readers (only the new result is read out).

A history item (one entry of qalc.history)

Field	Type	Contents
expr	String	The expression sent (after conversion)
result	String	The result as shown, with all its digits (a result over 450,000 bytes that the engine cut: its first 430,000 bytes and "…", the end in tail). fitHistory() adds up the lengths from the newest item; the items after the total reaches 1,000,000 characters (HISTORY_BUDGET) are cut to 2,000 characters (SHORT_RESULT) with "…" (1.2.0; before, 20,000 each)
approx	Boolean	true for an approximate value
cmp	Boolean	true for the result of a comparison or an equation (x = 2, true and the like)
base	Number	The number base of the display at the time
msgs	Array	Warnings and errors (at most 3, info left out; each {type, text}; text is the engine's English text, translated when it is shown on the Japanese screen). A result the engine cut gets one more warning, The middle of this long result is not shown (N digits)
t	Number	Time of the commit (ms)
input	String (optional)	The exact value to insert: the ans event's input, when it has at most 20,000 characters (MAX_INPUT). Old items beyond the budget drop an input over 2,000 characters and get big instead
cut	Boolean (optional)	result was cut (by the history budget or by the engine)
tail	String (optional)	The end of a long result that the engine cut (up to 20,000 bytes; 1.3.0). Shown as "beginning … end". Removed when the history budget shortens result
big	Boolean (optional)	input was too long to keep

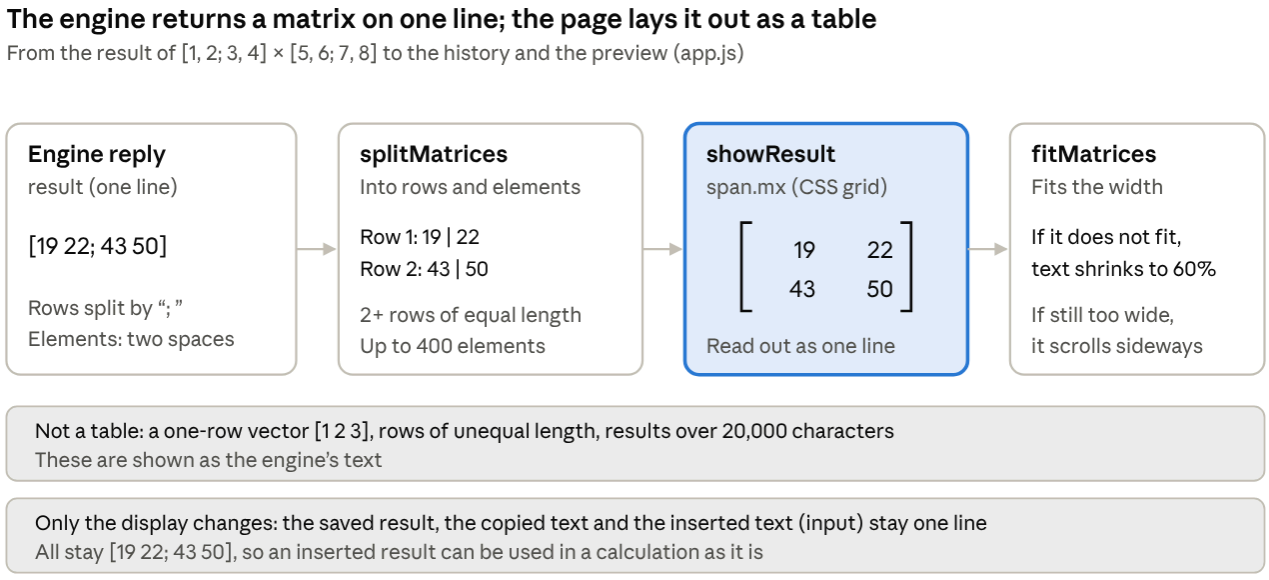
Matching the ans event

The engine sends the ans event with the same id right after the reply. The page remembers the items made since it started by id (sessionEntries); when the event arrives, it attaches input to the item and saves again. An event that arrives before its reply is kept (up to 20: earlyInputs) and used when the item is made.

How results are shown (displayText, displayResult)

- With digit grouping, the engine separates groups of digits with narrow spaces (U+202F in numbers of up to 20 characters, U+2009 in longer ones), in the decimals too, and puts U+202F between a number and its unit. The page shows only the groups of the integer part with "," (1,234,567) and leaves the decimals and the space before a unit as they are (3.141 592 654, 12.345 km). Since 1.3.0 no commas appear in the decimals.
- A long result that the engine cut (with a tail) is shown as "beginning … end", without the "…" at the end of the beginning (shownText). When the end starts in the middle of the decimals, its groups are left as they are.
- A result that is only a date ("2026-11-09" and the like) loses its quotes, and T becomes a space.
- Comparisons and equations are shown as they are; an approximate value gets "≈ " in front (not when the result already contains ≈), others "= ".

Matrices as tables (splitMatrices, showResult, fitMatrices)



Matrices as tables · from the engine's one-line text to a grid

- The engine returns a matrix as one line of text ([1 2; 3 4]: rows separated by "; ", the elements of a row by two spaces; an element that is more than one number or name is in parentheses). In the history and the preview, a matrix with two or more rows is drawn as a table.

- splitMatrices reads the result from the start and splits each “[” up to its matching “]” into rows and elements, following the nesting of brackets and strings in " ". Only a block with two or more rows, the same number of elements in every row, no empty element and at most 400 elements in all counts as a matrix. A one-row vector ([1 2 3]) and rows of different lengths stay text.
- To keep the display fast, it stops looking when the result is longer than 20,000 characters, has no “;”, or after 200 “[”.
- showResult makes each matrix a span.mx (a CSS grid whose number of columns is set in grid-template-columns) and puts the elements in one by one. The text around it (“= ”, “x = ”, operators between matrices) stays as it is. For screen readers it has role="img" and the original one-line text in aria-label.
- fitMatrices makes the text of a matrix wider than its place smaller (down to 60 %, at most 4 tries). A matrix that still does not fit scrolls sideways.
- Only the display changes: the result saved in the history, the text copied and the text inserted into an expression (input) stay one line.

Long results (fitResultBox, fitHistory, clip; 1.2.0 and 1.3.0)

- The highest precision is 200,000 digits (MAX_PRECISION, the same value in the page's app.js and the engine's qalc-engine.cc); without QCalc Pro it is 1,000 digits (FREE_MAX_PRECISION; 3.20). A number outside the range typed in the setting is brought into 2 to 200,000, and the engine keeps it in the same range.
- A result in the history (.h-res) taller than --res-max scrolls inside its box. fitResultBox sets --res-max to 70 % of the history's height (at least 96 px) and measures again with a ResizeObserver (resize where there is none). The history itself scrolls when moved outside the box.
- The heading of the long-press menu shows the first 300 characters of the expression and of the result, and the text for screen readers (#status) the first 200 of each (clip; “…” for the rest). “Copy result” in the menu copies all the digits.
- fitHistory runs after a commit, after the history changes and after the history is loaded at start. It adds up the lengths of result, tail, input and expr from the newest item; the items after the total reaches 1,000,000 characters get their result cut to 2,000 characters (cut) and their tail removed. The newest result is never cut. The saved JSON is at most about 2.1 million characters (up to about 1.5 million kept with all digits, about 0.6 million for 300 cut items; less than the 4 million that StateStore writes).
- A result the engine cut (truncated) is marked cut and is not inserted into expressions. sessionEntries (items waiting for their ans event) are dropped when their event arrives and kept to 20 (so that long results do not stay in memory).

- 1.3.0: the preview is calculated with the same max_chars 450,000 and tail_chars 20,000 as the commit and shows all the digits (before, the preview stopped at 6,000 bytes and looked different from the confirmed result). A result over 450,000 bytes (100000!, with 456,574 digits, for example) is shown in the preview and in the history as its first 430,000 bytes, "…" and its last 20,000 bytes, with the note "The middle of this long result is not shown (N digits)" (digits when they are at least 60 % of the characters, otherwise characters; cutMessage).

Using the history

Action	What happens
Tap an expression	It replaces the input field, with the cursor at its end (the expression that was being typed comes back with Undo; 3.8)
Tap a result	The result is inserted at the cursor (rules below)
Touch and hold (480 ms; cancelled by moving 10 px or more)	A menu: "Copy result", "Copy expression", "Edit expression", "Insert result", "Delete this line". The copied result has the same form as the screen (a matrix as one line). A deleted line comes back at its place with "Undo" in the notice "Line deleted" (5 seconds)
"Clear history" in the menu	After a confirmation, everything is cleared (the engine's ans stays). "Undo" in the notice "History cleared" (6 seconds) brings it back; calculations made meanwhile stay after the restored history. With an empty history, only "The history is empty" is shown

Where the history stays scrolled: while the history is at its newest item (within 24 px of the bottom), it keeps showing the newest line even when the input field or the preview grows and the history's height changes (ResizeObserver). When it has been scrolled up to read older items, it does not move.

The first screen (an empty history)

When there is no history, renderHistory shows in the history area how to start ("Type an expression and press =: the results stay here" and the like), four examples (STARTERS: 60 mph to km/h, 100 USD to EUR (100 USD to JPY on the Japanese screen), solve(x^2 − 5x +

6 = 0), diff(x^3)) and an “Examples and tips” button. A tapped example goes into the input field (setExpr), and the preview shows its result. This screen is read from the top, so it is scrolled to the top (with results, to the bottom, where the newest line shows).

What is inserted (insertText)

1. An item marked cut or big is not inserted: “This result is too long to insert (use the ans key)” is shown.
2. If there is an input, it is used. Examples: for “2 + 1/3 ≈ 2.333333333” on the screen, 7/3; for “0xFF” in hexadecimal, 255; for “x = 2 or x = −2”, [2 −2].
3. A history saved by an earlier version (no input) uses the shown text. With a base other than 10, 0x, 0b or 0o is added (duodecimal and Roman numerals are not inserted). For a comparison “name = value” only the value is used, and one with or or and is not inserted. Otherwise the part before “ ≈ ” or “ = ” is used.
4. A single number (with sign, decimal point and E exponent), a single name, an array in [ ] or a string in " " goes in as it is; anything else is wrapped in ( ) so that it does not bind to the operators around it.

3.10 Graphs (2D and 3D)

In both 2D and 3D, the engine (libqalculate) calculates the values and the page (a canvas) draws them and handles the gestures. Neither gnuplot nor WebGL is used.

2D graphs (y = f(x))

The 2D points are calculated with libqalculate's expressionToPlotVector.

Request from the page

Field	Value
exprs	Each line of the input field as one expression, at most 6 (empty lines left out; normalizeJa applied to each)
min / max	The text of the x-range fields (−10 and 10 at first; expressions such as pi allowed)
steps	600
var	x
timeout	12000 (shared by all expressions)
opts	The same settings as the calculator

In the engine (cmd_plot)

1. Without exprs, returns “exprs missing”; with more than 12, “too many expressions”. steps is kept in 10 to 4000 (300 when left out) and timeout in 100 to 120000 ms (8000 when left out).
2. Forces approximate calculation and, with a precision setting above 20 digits, calculates with 20 (the points are sent as doubles). The precision is set back afterwards.
3. Calculates min and max with a limit of 2 seconds each; when either is not a number, is nested too deeply, or min ≥ max, returns “invalid range”.
4. For each expression, calculates its points if 50 ms or more are left (timeout minus the time used). A y that is not a value becomes null, and a point whose x is not a value is skipped. A timeout, or an expression that is not a string, gives an empty series.
5. Returns series ({x:[…], y:[…]} per expression) and messages.

Range and ticks

- First view (autoView): x from the smallest to the largest point. y: with 50 points or more, from the value at the 2 % position to the one at 98 % (so that divergence such as tan(x) does not flatten everything); with fewer, from the smallest to the largest. A width of almost 0 is widened a little, and margins of 8 % are added above and below. With no points at all, −10 to 10.
- The tick spacing (niceStep) is chosen from 1, 2 and 5×10^n: about one line per 90 px across and per 70 px up (at least 3). Tick labels use as many digits as needed to tell neighbors apart, and very large or small values use E notation.

- Safety: with more than 400 lines, or positions beyond 2^{52}, no ticks are drawn (gridValues). After zooming or moving, a range is used only when its width is more than 10^{-12} of the size of its values and less than 10^{300} (viewOk); otherwise “Cannot zoom any further” is shown.

Drawing lines

- Six colors in turn, 2.2 px wide, drawn at the screen's pixel density (devicePixelRatio).
- A line breaks at a point whose y is null or infinite, and where the step to the next point is more than 1.5 times the screen height (so that no vertical line is drawn at an asymptote).
- The legend shows the expressions; one without any value gets “ (no values)”.

Gestures

Gesture	What happens
“Draw”	Saves the expressions and the range and calculates again for the range typed. Clears the mark and the coordinates of a point tapped on the previous graph
Drag with one finger	Moves the view
Pinch with two fingers	Zooms around the point between the fingers
+ / −	Zooms in / out 2 times around the center
⟲	Calculates again for the range typed and resets the view
Tap	Marks the nearest point and shows “expression: x = …, y = …” (10 significant digits). Clears it if no point is near

Calculating again outside the range (refetchIfNeeded)

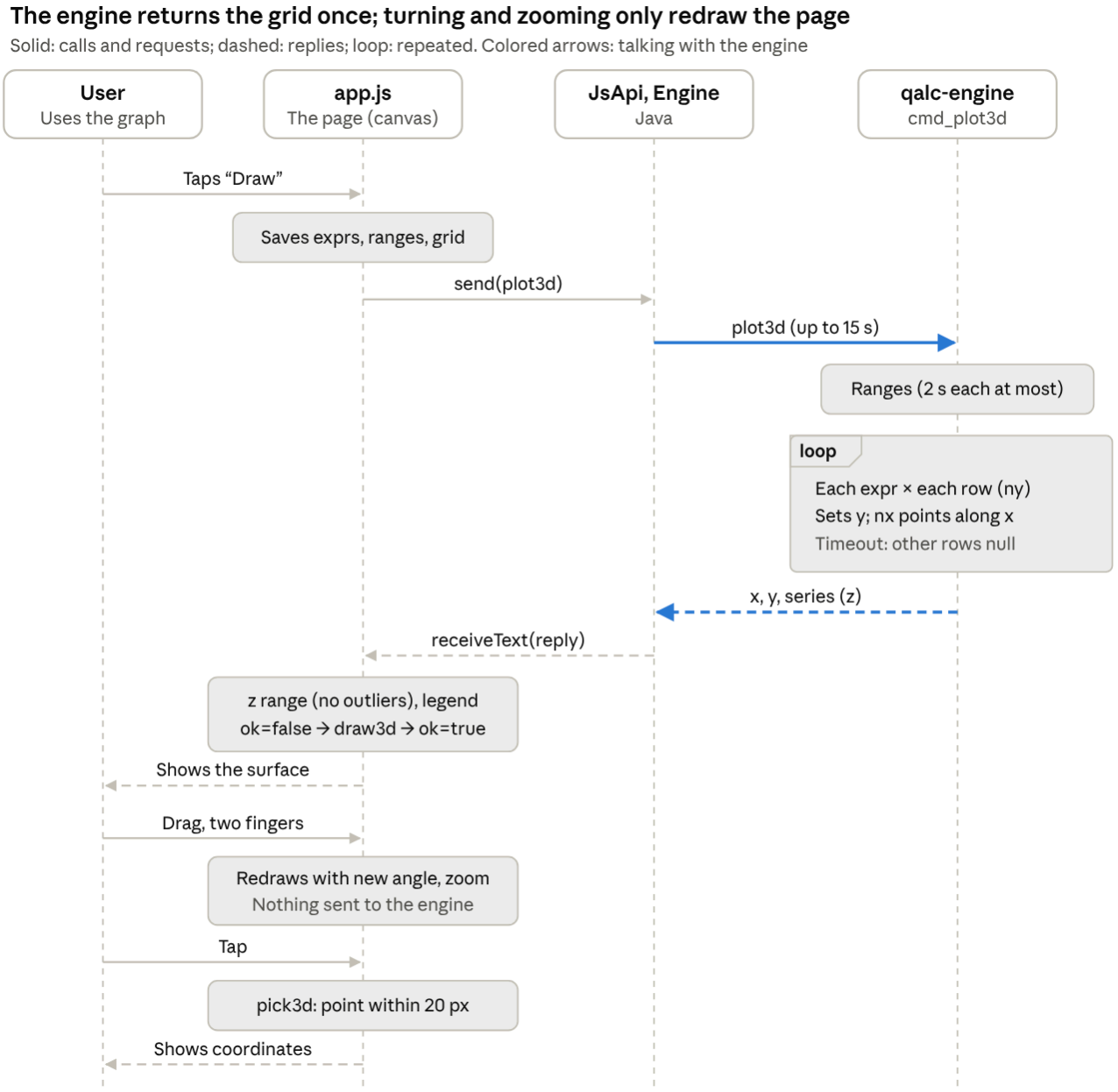
0.35 seconds after moving or zooming, when the view goes outside the calculated x range by 5 % of its width or more, or becomes narrower than 1/6 of the calculated width, the points are calculated again for the view plus half its width on each side (the view stays). Range numbers are sent with “e” written as “E” (a lowercase e can be read as Euler's number).

The last graph and a safe restart

The expressions and the range are saved in qalc.plot.exprs, min and max and drawn automatically next time. qalc.plot.ok is set to false before drawing and to true after it, so if the app stopped while drawing last time, nothing is drawn automatically and “Press ‘Draw’ to draw the graph” is shown. The error “invalid range” is shown as “Check the range of x”.

3D graphs (z = f(x, y))

The engine calculates z for all the grid points at once (plot3d), and the page draws the surface on a canvas. Turning, zooming and showing coordinates happen only in the page, with nothing sent to the engine. WebGL is not used: the quadrilaterals are painted from the back to the front (the painter's algorithm).



Sequence diagram · calculating and drawing a 3D graph

Request from the page (plot3d)

Field	Value
exprs	Each line of the input field as one expression, at most 4 (empty lines left out; normalizeJa applied to each). The variables are x and y
xmin, xmax, ymin, ymax	The text of the range fields (−5 and 5 at first; expressions such as pi allowed)
nx, ny	The “Grid” value: 25 (Coarse), 41 (Normal, the default) or 61 (Fine), the same for x and y
timeout	15000 (shared by all expressions)
opts	The same settings as the calculator

In the engine (cmd_plot3d)

1. Without exprs, or with none, returns “exprs missing”; with more than 4, “too many expressions”. nx and ny are kept in 2 to 121 (41 when left out) and timeout in 100 to 120,000 ms (15,000 when left out).
2. Forces approximate calculation, turns interval arithmetic off and allows complex numbers. With a precision setting above 20 digits it calculates with 20 (the values are sent as doubles) and sets it back afterwards.
3. Calculates xmin, xmax, ymin and ymax with a limit of 2 seconds each; when one is not a number, is nested too deeply, or the minimum ≥ the maximum, returns “invalid x range” or “invalid y range”.
4. x divides xmin to xmax into nx equal points (the last point is exactly xmax). y builds ny values by adding (ymax − ymin) ÷ (ny − 1) as libqalculate numbers.
5. For each expression, if 50 ms or more are left, sets a limit with startControl(time left) and then parses the expression (messages about typing mistakes are returned). The messages of single points are stopped; for each row, y is put in and generateVector calculates the nx points along x.
6. Only real values count (an imaginary part no larger than 10^{-12} of the real part counts as real). Complex, undefined, infinite and unit-bearing values become null. On a timeout, the remaining rows are filled with null and complete=false.
7. Returns x (nx values), y (ny values), series (z and complete per expression) and messages. Numbers are written with 12 digits.

Drawing (draw3d)

- Coordinates: x and y are fitted to −1 to 1, z to −0.72 to 0.72, in a box. The z range (zRange3d) goes from the smallest to the largest real value; but with 50 values or more, when the whole width is more than 3 times the width between the 1 % and 99 % points, it is the 1 % to 99 % range with 15 % of its width added as margins (so that poles such as 1/(x y) do not flatten the surface). Quadrilaterals with a grid point outside the range are not drawn.
- Projection (orthographic): from the azimuth az, the elevation el, the zoom and box coordinates (u, v, w): xr = u·cos az − v·sin az, yr = u·sin az + v·cos az, screen x = cx + xr·S, y = cy − (w·cos el + yr·sin el)·S, depth = yr·cos el − w·sin el, with S = min(W ÷ 3.3, H ÷ (2·(0.72·|cos el| + 1.42·|sin el|) + 0.75)) × zoom. At first az = −0.62, el = 0.52, zoom = 1.
- The box: the three faces at the back are tinted and get grid lines (about 4 steps from niceStep, at most 40 lines). Axes are drawn on the two front edges of the floor and the left vertical edge, with tick numbers and x, y and z (the z ticks at the floor's height are left out, as they would overlap the x and y ticks).
- Surface: for each grid quadrilateral, the sum of the depths of its four corners is found, and they are painted from the back. The shading is the absolute value of the dot product of the normal (the cross product of the two diagonals) and the light direction (−0.42, 0.62, −0.66) (from the top left, in front of the screen).
- Colors: with one expression, viridis colors by height (brightness 0.5 + 0.5 × shading); with two or more, one color per expression (brightness 0.42 + 0.58 × shading). Grids of 45 × 45 or less also get mesh lines. The legend shows a color bar with the smallest and largest z for one expression, and the color of each expression for two or more. An expression without any value gets " (no values)".
- Speed: the grid points are projected once per expression into typed arrays, which are reused at every redraw. Redraws while dragging are merged into one per frame with requestAnimationFrame.

Gestures (3D)

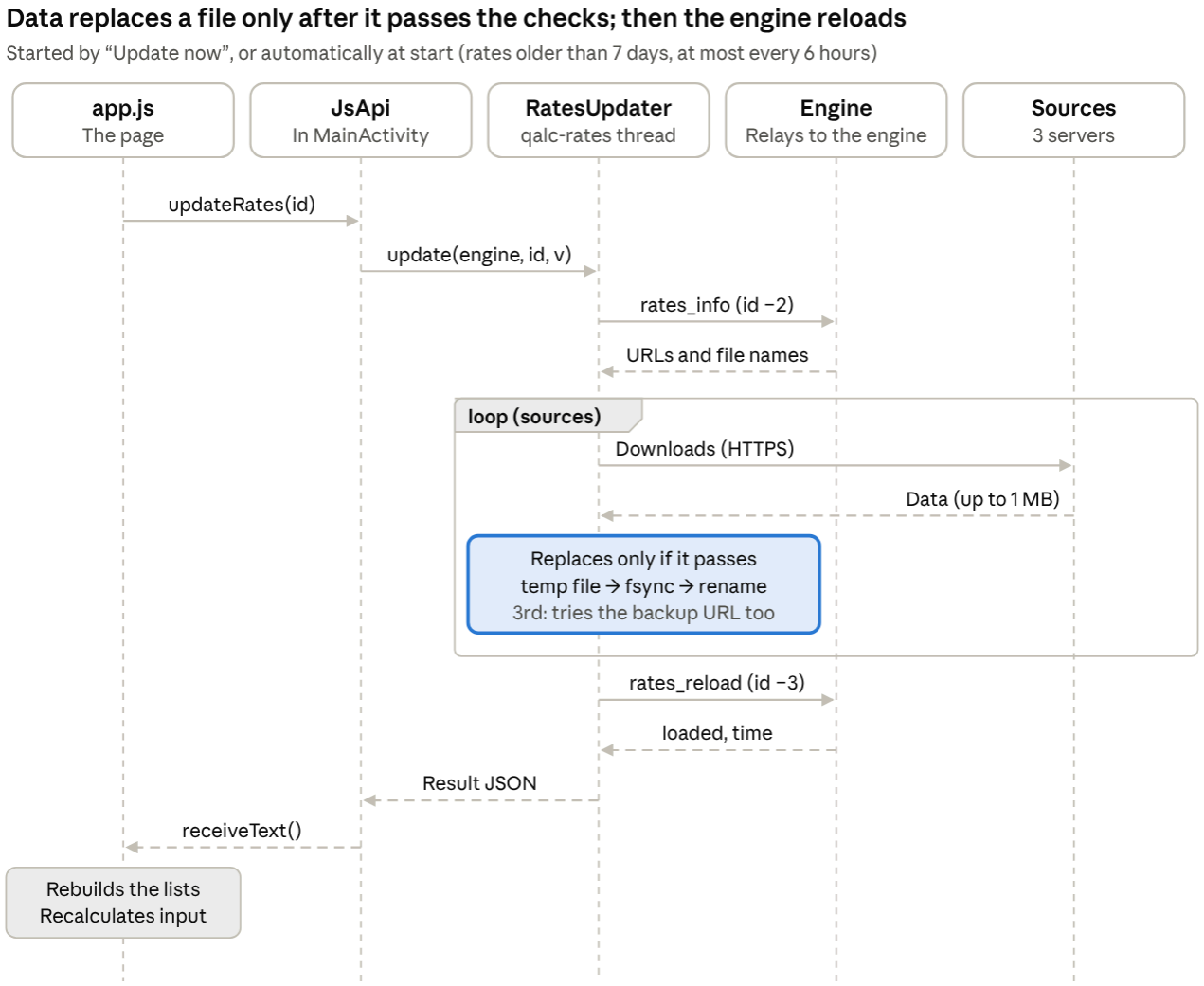
Gesture	What happens
"Draw"	Saves the expressions, the ranges and the grid and calculates again
Drag with one finger	Turns the surface: each px across changes the azimuth, and each px up or down the elevation, by 0.01 rad (the elevation up to ±1.5 rad)
Pinch with two fingers	Zooms (0.4 to 6 times)
+ / −	1.25 times / 0.8 times
⟲	Resets the direction and the size

Gesture	What happens
Tap	Marks the front-most grid point within 20 px of the finger and shows “expression: x = …, y = …, z = …” (10 significant digits). Clears it if none is near

Switching between 2D and 3D: the buttons at the top of the graph panel switch them. The kind shown last is saved in qalc.plot.mode, and the panel opens with it next time. The input and the picture stay when the panel is closed; the 2D graph is calculated when it is first shown. The 3D expressions, ranges and grid are saved in qalc.plot3d.*, and, as for 2D, the done mark qalc.plot3d.ok is changed before and after drawing, so after the app stopped while drawing, nothing is drawn automatically. When the screen size changes, both the 2D and the 3D graph are drawn again.

3D graphs are a QCalc Pro feature (1.3.0; 3.20). Without Pro, the 3D button carries a PRO mark, and pressing it shows the Pro prompt and keeps the 2D graph. A saved kind of 3d also opens as 2D (setMode).

3.11 Updating exchange rates



Sequence diagram · updating the exchange rates

The rates come from the same three sources as the official Qalculate! and are saved under the file names libqalculate uses. The engine is built without networking (no libcurl), so Java (RatesUpdater) downloads them, and the engine is only told to load the files afterwards. The URLs and file names are the ones libqalculate gives.

Sources and checks

No.	Source	File saved	Passes when
1	European Central Bank (https://www.ecb.europa.eu/stats/eurofxref/eurofxref-daily.xml)	eurofxref-daily.xml	It contains `<Cube` and `currency=` and ends with `</gesmes:Envelope>` (not cut off)
2	Coinbase (https://api.coinbase.com/v2/prices/spot?currency=EUR)	btc.json	It contains `"amount":` and `"currency":` and can be read as a JSON object

No.	Source	File saved	Passes when
3	fawazahmed0 currency-api (https://cdn.jsdelivr.net/npm/@fawazahmed0/currency-api@latest/v1/currencies/eur.json; on failure https://latest.currency-api.pages.dev/v1/currencies/eur.json)	rates.json	It contains `: {` and `"eur": 1,` (or the same without spaces or in capitals) and can be read as a JSON object

The checks are those of libqalculate's own downloader plus a check that the whole file arrived: given a cut-off file, libqalculate silently falls back to its old built-in rates.

Steps (RatesUpdater.update)

1. rates_info (id −2) gives the number, URL and file of each of the three sources.
2. Each source is downloaded over HTTPS: connect 15 seconds, read 20 seconds, up to 1,000,000 bytes. Redirects are followed. The User-Agent is QCalc/1.3.0 (Android; libqalculate 5.13.2), where 1.3.0 is the app's versionName (only letters, digits, "." and "-", up to 32 characters; 0 when unknown; 1.3.0-debug in the test build). Anything but HTTP 200 fails.
3. Data that passes the checks is written to a temporary file (.tmp), synced with fsync and renamed over the old file. On a failure the old file stays, and the source's host name goes into failed.
4. rates_reload (id −3) makes the engine load the files again. If 1 or more files were updated but loaded=false, the update fails.
5. The result (ok, updated, total, failed, time, and error on failure) goes to the page. The page shows when the rates were last updated and a result such as "Updated from 2 of 3 sources." (with "(Not available: …)" when some failed). On success it rebuilds the unit and currency lists and recalculates the expression being typed.

No two updates at once: update is synchronized, so automatic and manual updates never write the same file at the same time. The page disables the "Update now" button while it updates.

Automatic update: at start, when the setting "Update exchange rates automatically (every 7 days)" is on, init's rates_time is older than 7 days (or unknown) and 6 hours or more have passed since the last try, the rates are updated. On success "Exchange rates updated" is shown; on failure nothing is shown.

3.12 Saved data

All the app's data is in its private storage on the phone (/data/user/0/io.github.hiroshitsu.qcalc/; the test build has its own, io.github.hiroshitsu.qcalc.debug), which other apps cannot read. No data is sent to the

internet: getting exchange rates only downloads, and buying and checking QCalc Pro go through the phone's Google Play Store app (3.20).

Place (in the private storage)	Written by	Format	Contents
files/state/qalc.history.json	Page → StateStore	JSON array	The history (the newest 300; items in 3.9)
files/state/qalc.settings.json	Page → StateStore	JSON object	19 settings (text_size added in 1.1.0, lang and theme in 1.3.0; missing ones take their defaults. lang only mirrors the language in use, which qalc.lang decides)
files/state/qalc.lang.json	Page → StateStore	JSON string ("en" or "ja")	The language the user chose with "Language / 言語" (1.3.0; 3.21). English when missing
files/state/qalc.draft.json	Page → StateStore	JSON string	The expression being typed
files/state/qalc.plot.mode.json	Page → StateStore	JSON string ("2d", "3d")	The kind of graph shown last
files/state/qalc.plot.exprs.json, qalc.plot.min.json, qalc.plot.max.json	Page → StateStore	JSON string	The 2D expressions (one per line) and the x range
files/state/qalc.plot3d.exprs.json, qalc.plot3d.xmin.json, xmax.json, ymin.json, ymax.json, qalc.plot3d.res.json	Page → StateStore	JSON string	The 3D expressions (one per line), the x and y ranges and the grid ("25", "41", "61")
files/state/qalc.plot.ok.json, qalc.plot3d.ok.json	Page → StateStore	JSON boolean	Whether the last 2D and 3D drawing finished
files/state/qalc.rates.try.json	Page → StateStore	JSON number	When an automatic exchange-rate update was last tried (seconds)

Place (in the private storage)	Written by	Format	Contents
The WebView's localStorage	Page	The same keys and contents	A copy of everything above (for earlier versions and for checks on a PC)
shared_prefs/engine.xml (key ans)	Engine	String	The last ans (given back when the engine starts again)
shared_prefs/pro.xml	Pro	SharedPreferences	The state of QCalc Pro (1.3.0; 3.20): owned (bought), price (the price text), pre_pro (a build without billing was started), era (a build with billing was started), legacy (an earlier user), test_locked (the test build seen without Pro)
files/qalculate/definitions/variables.xml, functions.xml, units.xml, datasets.xml	Engine (libqalculate)	libqalculate's definitions XML	User-defined variables and functions
files/qalculate/broken-definitions/〈file name〉.〈time〉	Engine	As it was	Definitions files that could not be read at start, moved aside
files/qalculate/eurofxref-daily.xml, btc.json, rates.json	RatesUpdater	As the sources give them	Exchange rates (European Central Bank, Coinbase, fawazahmed0 currency-api)
The place the user chose (system file picker)	MainActivity	CSV (UTF-8 with a byte order mark)	An exported history (1.3.0; 3.23). Outside the private storage, so it stays when the app is removed

Example: the settings (defaults) and one history item

```
{"angle":"rad","approx":"auto","fraction":"auto","precision":10,"max_decimals
":-1,"base":10,"notation":"auto","units":"optimal","mixed_units":true,"comple
x":"rect","grouping":false,"structuring":"simplify","preview":true,"haptics":
true,"auto_ans":true,"auto_rates":true,"text_size":"normal","lang":"en","them
e":"blue"}
[{"expr":"7/3","result":"2 + 1/3 ≈
2.333333333","approx":false,"cmp":false,"base":10,"msgs":
[],"t":1794216542706,"input":"7/3"}]
```


Rules for writing

- StateStore (the page's state): a name has 1 to 64 characters from lowercase letters, digits, `.`, `_` and `-` (the page lowercases its keys and passes other characters as `_`). Each item holds up to 4,194,304 characters, and files over 8 MB are not read. One writer thread writes; content not yet written is replaced by newer content of the same name (reading prefers content waiting to be written). It writes a temporary file (.tmp), syncs it with fsync and then renames it, so a stop in the middle leaves the old content.
- The page reads from StateStore first and from localStorage when StateStore has nothing (earlier versions saved only to localStorage). It writes to both.
- ans: an empty value, or one over 500,000 characters, is not saved (it becomes null). It is written in the background with apply() of SharedPreferences.
- User definitions: all four files are written again at every change. The temporary file is made outside definitions/ (files/qalculate/.〈file name〉.tmp), because every file in definitions/ is loaded at start, then synced with fsync and renamed. A file that cannot be read as XML at start is moved to broken-definitions/, and the page is told with init's defs_error.
- Exchange rates: downloaded data is checked (3.11) and then replaced through a temporary file, fsync and a rename.

When data is removed

Action	What goes
“Clear history” in the menu	Only the history (ans, settings and definitions stay)
“Delete” in the user definitions panel	Only that variable or function
Clearing the app's data in Android's settings, or uninstalling	Everything

Action	What goes
Updating over it (from Google Play; for the test build, a new APK signed with the same test key)	Nothing

With allowBackup=true, this data can go into Android's backup, depending on the phone's backup settings. A purchase of QCalc Pro is recorded with the Google account, so after clearing the data or installing again it comes back with the same account when the purchases are asked for at start. The mark of an earlier user (legacy), though, is kept only on the phone and does not come back after the app's data is cleared (3.20).

3.13 Time limits and other limits

Time limits come in three layers. The innermost is the engine's own time limit, the next is the engine's watchdog thread, and the outermost are Java's waiting time and the page's last safeguard. When an inner one works, the outer ones are not used.

Request	Engine limit (timeout)	Watchdog thread (timeout + 3 s)	Java's wait (timeout × number of expressions + 10 s)	The page's last safeguard
Preview	1.5 s	4.5 s	11.5 s	120 s
Commit (=)	20 s	23 s	30 s	120 s
Graph (6 expressions)	12 s (shared by all expressions)	15 s	82 s	204 s
3D graph (4 expressions)	15 s (shared by all expressions)	18 s	70 s	180 s
Lists, definitions, init and others (no timeout)	–	–	30 s	120 s
Restoring ans (set_ans, id −1)	15 s	–	20 s	–
Updating exchange rates	–	–	30 s each for rates_info and rates_reload	300 s

The page's last safeguard is max(120 s, 2 × timeout × number of expressions + 60 s); a request without a timeout counts as 30 s.

Engine (qalc-engine.cc)

Item	Value
calc timeout	100 to 120,000 ms (3,000 when left out)
calc max_chars	100 to 10,000,000 bytes (200,000 when left out). What goes beyond is dropped, without cutting a character in two, and "…" is added. tail_chars is 0 to 1,000,000 bytes (0 when left out); the end of the result is returned only when twice tail_chars is no more than max_chars (1.3.0)
Length of parsed (the expression as understood)	2,000 bytes
plot timeout / steps / expressions	100 to 120,000 ms (8,000 when left out) / 10 to 4,000 (300 when left out) / up to 12
Calculating the plot range	2 s each for min and max. At least 50 ms must be left to start an expression
plot3d timeout / grid / expressions	100 to 120,000 ms (15,000 when left out) / nx and ny each 2 to 121 (41 when left out) / up to 4
Calculating the plot3d ranges	2 s each for xmin, xmax, ymin and ymax. At least 50 ms must be left to start an expression
plot3d values	Real values only (an imaginary part no larger than 10^{-12} of the real part counts as real), sent with 12 digits
Precision of plot and plot3d	Up to 20 digits (even when the setting is larger)
Nesting	Brackets (([{) up to 300 levels deep, and up to 1,000 ^
Watchdog check interval	0.2 s

Item	Value
Exit codes	0: the input was closed; 3: the watchdog answered after a timeout; 4: a restart after a forced stop; 5: an exception that cannot be handled (such as running out of memory)

Java (Engine, StateStore, RatesUpdater, MainActivity)

Item	Value
Length of the saved ans	Up to 500,000 characters (an approximate value with 200000 digits takes about 400,000)
One StateStore item	Up to 4,194,304 characters (files up to 8 MB are read)
Downloads	Connect 15 s, read 20 s, up to 1,000,000 bytes per file
Waiting for the page's answer to Back	0.8 s (with no answer, the app moves to the background)
Restarting the screen	When it stops twice within 15 s, a message is shown instead of restarting
Saving a file (saveFile)	A name of up to 100 characters from letters, digits, `.`, `_` and `-`, starting with a letter or a digit; the type text/csv; contents up to 16,777,216 characters
Asking about QCalc Pro	At start and each time the app comes back to the screen (onResume); also when the Pro panel opens

The page (app.js)

Item	Value
Wait before a preview starts	160 ms (max_chars 450,000, tail_chars 20,000)
max_chars and tail_chars when committing	450,000 and 20,000
Showing “Calculating… (tap here to stop)”	After 1.2 s

Item	Value
History	300 items. From the newest, results keep all their digits up to 1,000,000 characters in total; longer results older than that keep 2,000 characters. Up to 3 messages per item
Delay before saving	History 300 ms, the expression being typed 500 ms
ans events that arrive early, kept	20
Key repeat	From 420 ms, every 70 ms
Long press	480 ms (cancelled when the finger moves 10 px or more)
Vibration	8 ms
Precision setting	2 to 200,000 (up to 1,000 without QCalc Pro)
Automatic exchange-rate update	When the rates are older than 7 days; tried at most once every 6 hours
Graph	6 expressions; steps 600 (about 600 points; libqalculate adds or removes points); wait before calculating again 350 ms; up to 400 grid lines
3D graph	4 expressions; grid 25 × 25, 41 × 41 (the default) or 61 × 61; mesh lines when the grid is 45 × 45 or less; turning 0.01 rad/px (elevation ±1.5 rad); zoom 0.4 to 6 times; tap range 20 px; z outliers looked for with 50 values or more
Height of the input field	Up to 110 px × the text-size factor (Normal 1, Large 1.18, Extra large 1.38)
Notices (toasts)	2.2 s normally. With an “Undo” button: 5 s after deleting a line, 6 s after clearing the history
Matrix shown as a table	2 rows or more, up to 400 elements, when the result has 20,000 characters or less. Up to 200 “[” are looked at. Text shrinks to 60 % at most
Lists (functions, units, variables)	The first 400 items are shown. Searching starts 150 ms after typing stops
Expressions that Undo can bring back	1 (until the next typing, deleting or committing)

Item	Value
Exporting the history (CSV)	File name QCalc-history-YYYYMMDD.csv; columns Date, Expression, Result and Number (3.23)

3.14 Error codes and messages

Replies that Java makes (with code)

All have the form `{"id":…,"ok":false,"error":"…","code":"…"}`. The error text is in English (since 1.3.0; in Japanese before). On the Japanese screen the page shows each of them in Japanese through MESSAGES_JA. The Japanese texts are in 3.14 of the Japanese detailed design.

code	error text	When
bad_request	The request is not valid	The request contains a line break or cannot be read as JSON
timeout	The calculation took too long and was stopped	No reply within the waiting time (the engine is stopped and started again with the next request)
cancelled	The calculation was stopped	The user stopped it (the engine was stopped)
crash	The calculation engine stopped unexpectedly. Try a simpler expression.	The engine ended after the request was sent (the same request is not sent again)
engine	Communication with the calculation engine failed (reason)	Reading or writing failed after sending, or an unexpected exception happened in Java
engine	Could not start the calculation engine (reason)	Starting or sending failed twice in a row

Errors that the engine returns (ok=false)

Request	error text	Shown as	Meaning
All	bad request (id is null)	As it is (in English on the Japanese screen too)	Not readable as JSON, not an object, or extra characters after it

Request	error text	Shown as	Meaning
All	unknown command	As it is (in English on the Japanese screen too)	cmd is not known
All	engine error: 〈reason〉 (restart:true)	As it is (in English on the Japanese screen too)	An exception happened while processing. The engine ends after replying
All	Not enough memory for the calculation (restart:true)	As it is (in Japanese on the Japanese screen)	An exception that cannot be handled happened in the calculation thread or elsewhere (exit code 5)
calc, set_ans, define	The expression is nested too deeply	As it is (in Japanese on the Japanese screen)	Beyond the nesting limits in 3.13
set_ans	expr missing	Not shown (the page and Java send this request themselves)	expr is missing (when it cannot be calculated: ok=true, set=false)
plot	exprs missing / too many expressions / invalid range	invalid range as “Check the range of x”; the others as they are	No array of expressions / more than 12 / the range is not a number, or min ≥ max
plot3d	exprs missing / too many expressions / invalid x range / invalid y range	invalid x range as “Check the range of x”, invalid y range as “Check the range of y”; the others as they are	No array of expressions, or an empty one / more than 4 / the x or y range is not a number, or the minimum ≥ the maximum
catalog	unknown catalog kind	As it is (in English on the Japanese screen too)	kind is none of functions, variables and units

Request	error text	Shown as	Meaning
define_var, define_func	Enter a name and a value	As it is (in Japanese on the Japanese screen)	The name or the value is empty
define_var, define_func	Check the value (libqalculate's message)	As it is (in Japanese on the Japanese screen, with common libqalculate messages translated too)	The value cannot be read as an expression (an error or a warning came)
define_var	The variable name contains characters that cannot be used	As it is (in Japanese on the Japanese screen)	It does not follow libqalculate's rules for variable names
define_func	The function name contains characters that cannot be used	As it is (in Japanese on the Japanese screen)	It does not follow libqalculate's rules for function names
define_var, define_func	"name" is a built-in name and cannot be used	As it is (in Japanese on the Japanese screen)	The same name as a built-in variable, unit or function
define_func	"to" cannot be used in the expression of a function	As it is (in Japanese on the Japanese screen)	The expression contains a conversion (to)
delete_def	There is no user definition "name"	As it is (in Japanese on the Japanese screen)	No user definition has that name

A timeout is not a failure: the reply has ok=true and timeout=true (and restart=true when the watchdog answered). When saving the definitions fails, the reply also has ok=true, with saved=false; the page then shows “Set, but it could not be saved to a file”.

Updating exchange rates (replies of RatesUpdater)

Situation	Reply
1 or more were updated	ok=true, updated (the number updated), total (the number of sources), failed (the host names of the sources that failed), time
None could be downloaded	ok=false, with the error “Could not download the exchange rates. Check the network connection.”

Situation	Reply
The engine could not load the new files, or a similar problem	ok=false, with the error “Could not update the exchange rates (reason)”, for example with the reason “The calculation engine could not load the new rates”

On the Japanese screen these texts are shown in Japanese.

Messages that the page shows

Each text of the page has an English and a Japanese version, chosen with tr(English, Japanese) (3.21). The table shows the English texts; the Japanese ones are in 3.14 of the Japanese detailed design.

Message	When
The calculation engine is starting	= was pressed while the engine was starting
(the error) (tap here to try again)	init failed. A tap starts the engine again
No reply from the calculation engine	The page's last safeguard ran out (code=timeout)
Could not read the reply of the calculation engine	A reply cannot be read as JSON (the oldest waiting request fails)
Check the expression	The result is empty and there is no error message
This result is too long to insert (use the ans key)	A result shortened because of its length, or one like it, was tapped
Check the range of x / Could not calculate the graph	The range of the graph is not valid / other failures
Check the range of y	The y range of the 3D graph is not valid
The calculation took too long: only part of the graph was calculated	The 3D graph did not finish within its time limit (the rows calculated are drawn)
There are no real values (check the ranges of x and y)	All the 3D values are complex, undefined or the like
Enter a function	“Draw” was pressed with no 3D expression
Press "Draw" to draw the graph	The drawing of the previous graph did not finish (it is not drawn automatically)

Message	When
Could not load the license texts.	licenses.js could not be loaded (3.16)
Cannot zoom any further	The view of the graph reached its limit (3.10)
The saved user definitions could not be read, so their file was moved aside	init came with defs_error
QCalc needs Android System WebView, which is not available. Update it in the Play Store or turn it on in the Settings app, then open QCalc again.	The WebView cannot be made (MainActivity)
The screen stopped again and again. Close the app and open it again.	The screen stopped twice within 15 s (MainActivity)
Line deleted / History cleared (both with an “Undo” button)	A line of the history was deleted / the history was cleared (3.9)
The history is empty	“Clear history” or “Export history (CSV)” was chosen in the menu with no history
Could not load the list (reason)	The list of functions, units or variables could not be fetched (a “Try again” button is shown)
Showing the first 400 items. Search to narrow them down.	More than 400 items match
The middle of this long result is not shown (N digits) / (N characters)	The result is longer than 450,000 bytes (in the preview and the history; 3.9)
“… is a QCalc Pro feature” (a precision above 1000 digits, 3D graphs, color themes, exporting the history) with an “About QCalc Pro” button	A Pro feature was used without Pro (3.20)
QCalc Pro is unlocked. Thank you!	Pro became available, by a purchase or otherwise
Checking your purchases…	“Restore a purchase” was pressed in the Pro panel

Message	When
Google Play is not available on this device / Could not connect to Google Play. Try again later. / QCalc Pro cannot be bought right now / No purchase of QCalc Pro was found / The purchase could not be completed (code) / The payment is pending. QCalc Pro will be unlocked when it is complete.	PlayBilling reported it in the message of a pro event (3.20)
History exported / Could not save the file	Exporting the history finished / failed (3.23; nothing is shown when the file picker was closed without choosing a place)
Copied (Android 12 and earlier) / Too long to copy	The text was put on the clipboard / the clipboard did not take it (MainActivity; 3.2)

Japanese versions of libqalculate's messages (MESSAGES_JA)

libqalculate's messages are in English (it has no Japanese translation). On the Japanese screen, MESSAGES_JA (57 entries) turns common ones into Japanese, and the others are shown in English. The 57 are the 28 libqalculate texts in the table below (the 14 about parsing and matrices were added in 1.1.0), the 19 texts of Java and the engine (the tables above), 6 Google Play texts (3.20), 1 about saving a file (3.23) and 3 notes about long results (3.9). On the English screen, all of them are shown in English as they are. The preview shows the first error (or, when there is none, the first warning); a committed result shows up to 3 messages, leaving out info.

libqalculate's message (English)	Shown on the Japanese screen
It has been N days since the exchange rates last were updated.	為替レートが N 日間更新されていません（メニュー→「為替レートの更新」）
Misplaced operator(s) "X" ignored. / Misplaced 'X' ignored.	余分な演算子「X」を無視しました / 余分な「X」を無視しました
Division by zero.	ゼロで割っています
Definite integral was approximated.	定積分は数値計算による近似値です
Empty expression in parentheses interpreted as zero.	空のかっこは 0 として計算しました
A global unit or variable was deactivated. …	同じ名前の組み込みの単位・変数を一時的に置き換えました（新しい変数を削除すると元に戻ります）

libqalculate's message (English)	Shown on the Japanese screen
You need at least N argument(s) (A) in function F().	F() には引数が N 個以上必要です（A）
Did you mean "A" (instead of "B")?	「B」ではなく「A」のことですか？
limit() is incomplete and unreliable.	limit() は完全ではなく、結果が正しくない場合があります
No unknown variable/symbol was found.	解く対象の未知数（変数）が見つかりません
Conversion of only part of an expression is not supported.	式の一部だけを変換することはできません
Unable to find a solution.	解が見つかりませんでした
Unable to integrate the expression. / Unable to integrate the expression exactly.	この式は積分できませんでした / この式は厳密には積分できませんでした
Unable to find limit.	極限を求められませんでした
No differential equation found.	微分方程式が見つかりません（例：dsolve(diff(y, x) = y)）
Unable to solve differential equation.	この微分方程式は解けませんでした
To avoid division by zero, the following must be true: C.	ゼロで割らないための条件：C
Was unable to isolate X with the current assumptions. The assumed (type and) sign was therefore temporarily set as unknown.	X を実数として求められなかったため、その仮定を外して解きました（複素数の解を含むことがあります）
Was unable to completely isolate X. / Was unable to isolate X.	X について完全には解けませんでした / X について解けませんでした
Inverse of singular matrix.	逆行列がありません（行列式が 0 の行列です）
Please use the cross(), dot(), and hadamard() functions for vector multiplication.	ベクトルどうしの積は、cross()（外積）、dot()（内積）、hadamard()（要素ごとの積）を使ってください
The second matrix must have as many rows (was R) as the first has columns (was C) for matrix multiplication.	行列の積は、左の行列の列の数（C）と右の行列の行の数（R）が同じでなければ計算できません
Argument N, A, in F() must be a square matrix.	F() の N 番目の引数は正方行列（行と列の数が同じ行列）でなければなりません

libqalculate's message (English)	Shown on the Japanese screen
F() requires that all matrices/vectors have the same dimensions.	F() に渡す行列・ベクトルは、すべて同じ大きさでなければなりません

3.15 The window and Back (Android 13 to 16)

QCalc targets SDK 36, so on Android 15 and later the page extends behind the system bars, and on Android 16 the old onBackPressed() is not called. MainActivity's setUpWindow() handles both without changing how older versions behave.

The window's containers (setUpWindow)

- On the root FrameLayout (in the color of the navigation bar), a View that paints behind the status bar (statusScrim) and a FrameLayout that holds the page (content) are stacked. The WebView and messages go into content.
- Android 15 and later (API 35+): an OnApplyWindowInsetsListener on the root View turns the insets of systemBars and displayCutout into padding of content; the bottom padding is the larger of the navigation bar and the keyboard (ime). statusScrim is as tall as the status bar. The insets are used up here (CONSUMED).
- Android 14 and earlier: there is no insets handling, and the system lays out the window as usual (with windowSoftInputMode="adjustResize", the page shrinks by the height of the keyboard).

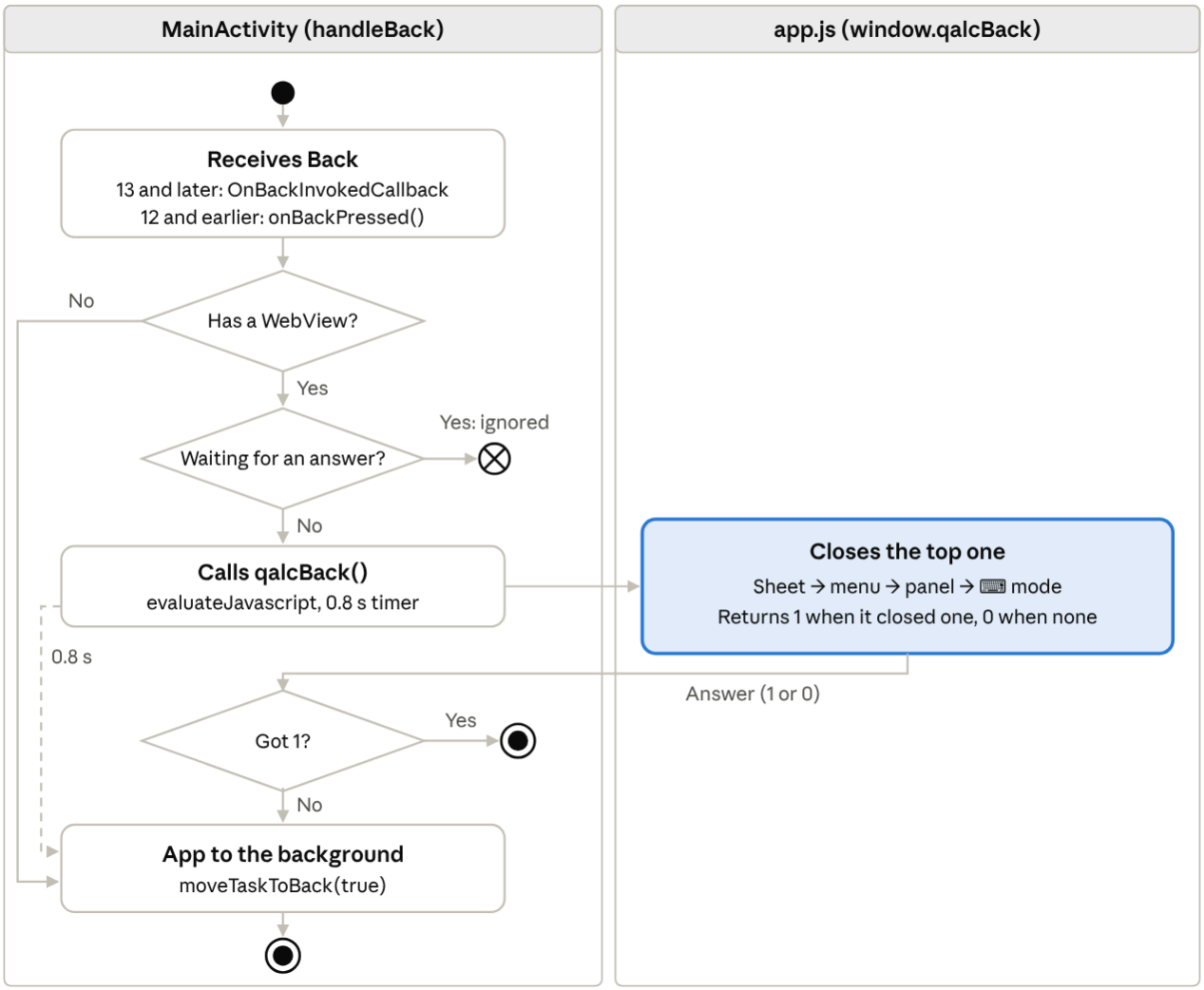
Colors of the system bars (applySystemBarColors)

Android	Status bar	Navigation bar	Light or dark icons
15 and later (API 35+)	Transparent, showing the color of statusScrim behind it (light #FFFFFF, dark #1C1F26, as the top bar)	Transparent, showing the color of the root View (light #F6F7F9, dark #121418). setNavigationBarContrastEnforced(false) keeps the system from adding a translucent scrim	WindowInsetsController
11 to 14 (API 30 to 34)	setStatusBarColor (the same colors)	setNavigationBarColor (the same colors)	WindowInsetsController
8.0 to 10 (API 26 to 29)	setStatusBarColor	setNavigationBarColor	SYSTEM_UI_FLAG_LIGHT_STATUS_BAR, LIGHT_NAVIGATION_BAR
7.0 and 7.1 (API 24 and 25)	setStatusBarColor	setNavigationBarColor (#121418 in the light theme too: these versions cannot show dark icons on a light navigation bar)	SYSTEM_UI_FLAG_LIGHT_STATUS_BAR

Switching dark mode re-creates the Activity (configChanges does not include uiMode), and the colors are chosen again.

Back (handleBack)

Back closes the top layer; with nothing open, the app moves to the background



Activity diagram · deciding what Back does

- Android 13 and later (API 33+): an OnBackInvokedCallback (PRIORITY_DEFAULT) is registered with the OnBackInvokedDispatcher, and the manifest sets android:enableOnBackInvokedCallback="true". Android 12 and earlier use onBackPressed(). Both call handleBack().
- handleBack() calls the page's window.qalcBack() with evaluateJavascript. The page closes the top layer (sheet → menu → panel → keyboard input mode) and returns 1, or returns 0 when nothing is open.
- On 0, moveTaskToBack(true) moves the app to the background (the process and the expression being typed stay; this is what the system does by default on Android 12 and later).
- When no answer comes within 0.8 s (the page is busy), the app moves to the background. A second Back while waiting for the answer is ignored (backPending). Without a WebView (it could not be made, or its renderer stopped), the app moves to the background at once.

- Because the callback is always registered, the “back to home” animation of predictive back is not shown, and the steps above happen when the finger is lifted.

3.16 Open source licenses

The copyright notices and full license texts of the app itself (QCalc) and of the seven pieces of software built into it (8 in all) are shown under About → Open source licenses. make-licenses.py collects the texts from the license files of the original sources into app/assets/licenses.js (about 90 KB).

- licenses.js puts into window.QCALC_LICENSES: components (name, version, license, copyright, url and texts of each), titles (the names of the license texts) and texts (the full texts, each only once).
- It is loaded with a script element only when the panel is first opened (so that starting is not slowed). If it cannot be loaded, “Could not load the license texts.” is shown.
- Each component gets a card. Its license texts go in details elements and are filled in only when opened (so that long texts are not laid out from the start). Text is put in with textContent and is never parsed as HTML.
- When the version of a library changes, run make-licenses.py again after fetch-sources.sh (Build and maintenance manual 5.9).

Component	Version	License	Copyright notice	Full texts included (read from)
QCalc	1.3.0 (not shown on the license screen; the version is in About)	GNU GPL v2 or later	Copyright (C) 2026 HiroshiTsu	GPL v2
libqalculate	5.13.2	GNU GPL v2 or later	Copyright (C) 2003-2007, 2008, 2016-2024 Hanna Knutsson	GPL v2 (libqalculate's COPYING)
GNU MP (GMP)	6.3.0	GNU LGPL v3 or later, or GNU GPL v2 or later	Copyright 1991, 1993-1997, 1999-2016, 2020, 2021 Free Software Foundation, Inc.	LGPL v3, GPL v3, GPL v2

Component	Version	License	Copyright notice	Full texts included (read from)
GNU MPFR	4.2.2	GNU LGPL v3 or later	Copyright 1999-2025 Free Software Foundation, Inc.	LGPL v3, GPL v3 (MPFR's COPYING.LESSER and COPYING)
libxml2	2.15.4	MIT	Copyright (C) 1998-2012 Daniel Veillard. Copyright (C) The Libxml2 Contributors.	libxml2's Copyright
musl libc	Comes with Zig 0.17.1	MIT	Copyright © 2005-2020 Rich Felker, et al.	musl's COPYRIGHT (zig package)
LLVM libc++ / libc++abi / libunwind	Comes with Zig 0.17.1	Apache License v2.0 with LLVM Exceptions	LLVM Project contributors	libcxx's LICENSE.TXT (zig package)
Zig compiler_rt	0.17.1	MIT	Copyright (c) Zig contributors	Zig's LICENSE

Because MPFR is under LGPL v3 or later, the calculation engine (libqalcengine.so) as a whole is distributed under GNU GPL v3 or later (v3 is the version that can be combined with libqalculate's GPL v2 or later). The corresponding sources are in third_party/ of the repository, which can be reached from "QCalc source code (GitHub)" in the app.

QCalc's own code (everything outside third_party/) carries an exception that allows linking it with the Google Play Billing Library and the libraries that library uses, and distributing the result (License in the repository's README; 1.3.0). The exception does not extend to libqalculate or the other GPL software, but the app's code does not link with libqalculate: libqalculate runs as a separate program (the calculation engine).

3.17 App Bundle layout and signing

The App Bundle for Google Play is put together by build-aab.sh, without Gradle or bundletool. The .aab is checked with the validate command of bundletool 1.17.2, and building device APKs from it is tested too (chapter 6).

Path	Contents	Made by
BundleConfig.pb	The bundletool version (1.17.2), and the “uncompress native libraries” optimization turned off	Python in build-aab.sh writes the protobuf directly
base/manifest/AndroidManifest.xml	package io.github.hiroshitsu.qcalc, versionCode 5, versionName 1.3.0, minSdk 24, targetSdk 36 (protobuf format)	aapt2 link --proto-format
base/resources.pb, base/res/	App name, theme and icons (the resource table in protobuf format)	aapt2 compile and link (the API 34 android.jar)
base/assets/	The page (index.html, style.css, app.js, licenses.js)	As they are
base/dex/classes.dex	The 5 Java files (app/src; PlayBilling is not included), compiled	javac (the API 36 android.jar, Java 8) → dalvik-exchange
base/lib/arm64-v8a/libqalcengine.so	The calculation engine (7,588,488 bytes, a statically linked executable)	build-engine.sh
META-INF/	The signature of the upload key (SHA-256 with RSA)	jarsigner

- aapt2 links with the API 34 android.jar, because Ubuntu's aapt2 cannot read the resource tables of android.jar for API 35 and later (the resources used all exist in API 34). Java is compiled against the API 36 android.jar and refers to newer APIs such as OnBackInvokedCallback and WindowInsets directly.
- BundleConfig sets uncompress_native_libraries to enabled: false. By default, bundletool stores .so files uncompressed in the APKs it makes and sets extractNativeLibs to false, so the engine would not be extracted as a file and could not be run. With the option off, the engine in the APK is compressed (7.6 MB → about 3.1 MB) and extracted to nativeLibraryDir when the app is installed.
- 16 KB pages: the engine's 4 loadable segments (LOAD) are all aligned to 0x4000 (16 KB), so it runs on devices with 16 KB pages. Google Play requires apps that target Android 15 and later to support 16 KB pages.

- Signing: jarsigner (-digestalg SHA-256, -sigalg SHA256withRSA) adds the signature of the upload key signing/qcalc-upload.jks (alias upload), and jarsigner -verify checks it. The password comes from the environment variable QCALC_UPLOAD_PASS or the first line of signing/qcalc-upload.pass. Google Play checks this signature and then delivers APKs signed with the app signing key.
- Versions: build-apk.sh, build-aab.sh and Gradle read VERSION_NAME and VERSION_CODE from version.properties. VERSION_CODE goes up for every upload to Google Play.
- The time of each ZIP entry is fixed at 1981-01-01, so the same input gives the same content apart from the signature.
- PlayBilling, used to buy QCalc Pro, needs the Google Play Billing Library, so it is not in the App Bundle of build-aab.sh (billing is then none, and all the Pro features can be used). An App Bundle in which Pro can be bought is built with Gradle and qcalc.billing=true (Build and maintenance manual; 3.20).

3.18 Examples, lists and searching in Japanese

Version 1.1.0 added the examples panel, so that people who do not remember how to write an expression can still use QCalc, and made the lists of functions, units and variables searchable in Japanese. Since 1.3.0 the screen is in English or Japanese, so the descriptions of the examples and the category names of the lists are shown in the language of the screen, and the Japanese names and searching in Japanese are used only on the Japanese screen.

Examples and tips (openExamples, useExample)

- EXAMPLES is an array of “a category name and a list of [expression, description]”: 12 categories with 69 examples (Basics 9, Unit conversion 8, Currency conversion 4, Equations 4, Derivatives 6, Integrals 6, Limits, series and differential equations 7, Matrices 8, Vectors 4, Number bases and integers 5, Dates 3, Statistics, uncertainty and constants 5). TIPS holds 11 tips.
- Tapping an example closes all open panels and puts the expression into the input field (useExample → setExpr). The expression that was being typed can be brought back with “Undo”. The preview calculates it, and nothing goes into the history until = is pressed.
- The examples are written with the characters the keys type (×, −, π). All the examples and the 4 on the first screen are run through the engine in the tests, which check that each gives a result without errors (73 expressions; Test specification and results). On the English screen 3 currency examples differ (100 USD to EUR, 50 EUR to GBP and 1 BTC to USD; the example on the first screen is 100 USD to EUR too), and these 3 were also run through the engine and give results without errors (1.3.0).

- The panel opens from 3 places: “Examples and tips” in the menu, the button on the first screen, and the link in About.

Loading the lists (loadCatalog)

- The catalog command fetches the functions, variables and units, which are kept per kind. They are dropped when user definitions change or the exchange rates are updated, and fetched again when a list next opens (invalidateCatalogs).
- Items that cannot be used in this app are left out (HIDDEN_ITEMS: the functions plot, command, load and export, and the variable uptime). Of the engine's 425 functions, 421 are shown.
- On the Japanese screen, each item gets a Japanese name and a Japanese category name (CATEGORY_JA). On the English screen they are not added, and items are shown and searched by their English names and categories. The Japanese names come from FUNC_JA for functions (230), VAR_JA for variables and constants (61) and JA_NAMES for units and currencies (124, looked up by the unit's name or abbr).
- The text searched joins the name, abbr, display name, English title, description, category and the Japanese name (on the Japanese screen only), in lowercase.
- The tables (FUNC_JA and the others) are looked up with names from outside (such as variable names the user gave), so own() is used to avoid values inherited from Object.prototype (such as constructor).

Searching and order (openCatalog)

- Only the items that contain the text of the search field (in lowercase, without spaces at either end) are shown. The category buttons (“All” and the top-level categories) narrow them down too. The unit list first shows “Currency”.
- While searching, items are ordered by how well they match: ① the name, abbr or Japanese name is exactly the text; ② the name or title starts with the text, or the Japanese name contains it; ③ the name or title contains it; ④ only the description or the category contains it. Within a rank, items follow the order of their titles. For example, in the function list “min” puts Min first (and on the Japanese screen, 積分 puts Integrate (integrate) first).
- At most the first 400 items are shown. The number of matches is shown as “n items”.
- openCatalog(kind, category) with a category clears the search and opens with that category chosen (“more…” on the Calculus keys opens Calculus, and on the Matrix keys, Matrices & Vectors). The row of category buttons scrolls so that the chosen one can be seen.

Rows and inserting (catalogRow, useItem)

List	Shown	Buttons
Functions	Title (and the Japanese name), how to write it (syntax), category, description	“Insert”: puts “name()” and places the cursor inside the brackets
Units and currencies	Title (and the Japanese name), the text to write in an expression (abbr; when it differs from the internal name, that name follows in brackets; compound units do not show the internal name), category, description	“Convert”: puts “ to text”. “Insert”: puts the text (not shown for units with insert=false)
Variables and constants	Title (and the Japanese name), name (display name and value), category, description	“Insert”: puts the name

Every button closes all open panels and then inserts at the cursor of the input field. Tapping a row opens or closes its whole description.

The text that writes a unit in an expression (the engine's unit_input_text and reads_back_as)

- Compound units (km/h, kW·h and others) have internal names such as km_p_h and kW_h in libqalculate, and those names are not read as units when written in an expression. So the list uses the form in which results show them (short names, Unicode symbols, prefixes). Other units use the name meant for input (preferredInputName).
- Whether the text is really read as that unit is checked: “3 text” is calculated and compared with 3 × the unit, both converted to base units (for example, Ha/e is read as “hartree divided by Euler's number e”, so it fails). On a failure the internal name is tried too, and if that cannot be read either, insert=false is returned (the unit can still be a conversion target).
- Currencies are not checked: their names are only currency codes or symbols, and the checking calculation would use up the warning that the exchange rates are old (shown only once).
- Texts that pass are remembered. Failures are tried again the next time the list is fetched (a user variable m = 3 may have hidden the unit m, for example).

3.19 Additions to the calculation engine, and workarounds for libqalculate problems

libqalculate's source is not changed. The bridge program (qalc-engine.cc) does the following three things to the running engine. The added units and function are not saved in the user's definition files; they are added each time the engine starts. Apart from these, the results are the same as those of the official qalc (309 expressions compared; Test specification and results).

Japanese units (add_japanese_units)

Unit	Reading	Kind	Definition (exact)	About
坪	tsubo (tsubo also works in expressions)	Area	400/121 m² (1 ken × 1 ken)	3.3058 m²
反	tan	Area	120000/121 m² (300 tsubo)	991.74 m²
寸	sun	Length	1/33 m (1/10 shaku)	3.0303 cm
尺	shaku	Length	10/33 m	30.303 cm
間	ken	Length	20/11 m (6 shaku)	1.8182 m
里	ri	Length	43200/11 m (2160 ken)	3.9273 km
合	gō	Volume	2401/13310 L (1/10 shō)	180.39 mL
升	shō	Volume	2401/1331 L	1.8039 L
匁	monme	Mass	15/4 g	3.75 g
貫	kan	Mass	3750 g (1000 monme)	3.75 kg

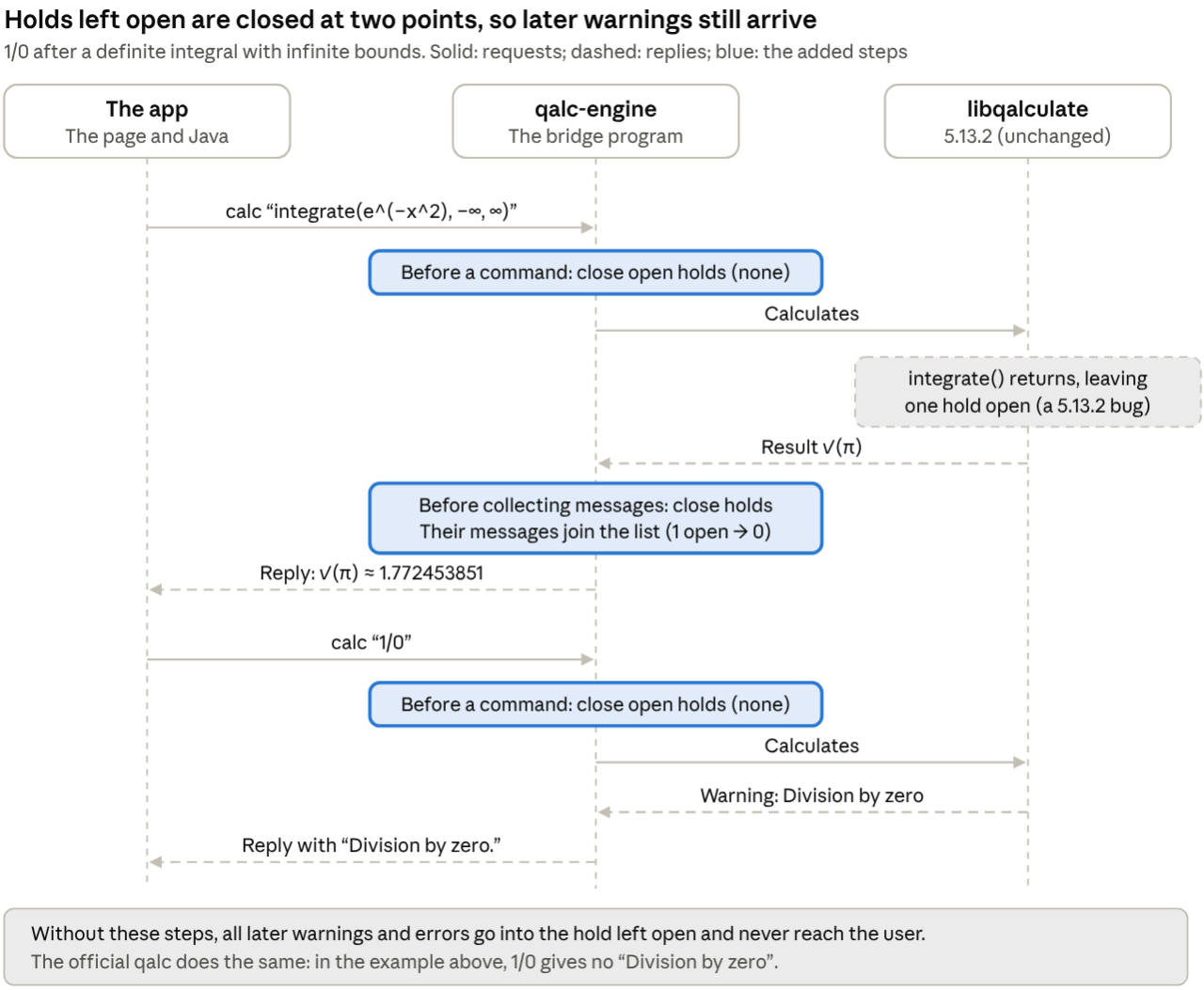
- The values are exact fractions from the definitions of the Weights and Measures Act of 1891 (1 shaku = 10/33 m, 1 shō = 2401/1331 L, 1 kan = 3.75 kg); for example, 30 坪 to m² ≈ 99.17355372 m².
- They are added as libqalculate AliasUnits in the category Japanese Units (shown as 日本の単位 in the Japanese list). The only Latin name is tsubo for 坪. Short names such as ken, go and tan are not added, as they might be read as something else in an expression.

- A unit is not added when a unit or variable of the same name already exists (for example, if a future libqalculate version has the same unit).

Matrix trace: trace() (TraceFunction)

- libqalculate has det() (determinant), adj() (adjugate), rk() (rank), cofactor() and others, but no trace (the sum of the diagonal elements). A function trace with one argument (a MatrixArgument that requires a square matrix) is added in the category Matrices & Vectors.
- It adds the diagonal elements `m[i][i]` in turn (calculateAdd), stopping if an abort is requested. For an argument that is not a square matrix, libqalculate gives its “must be a square matrix” error.
- Examples: `trace([1, 2; 3, 4])` = 5. Matrices with symbols work too (`trace([x, 1; 2, y])` = x + y).
- Of the matrix calculations libqalculate lacks, eigenvalues can be found with existing functions (solve(det(M − x × identity(n)) = 0), one of the examples). rank() is a different function that returns ranks (positions in order); the rank of a matrix is rk().

Warnings and errors that stop appearing, and the fix (close_message_stops)



Sequence diagram · a message hold left open, and the fix

- The problem: libqalculate holds back the warnings and errors raised during a calculation in a “message hold” (opened with beginTemporaryStopMessages and closed with endTemporaryStopMessages), and holds can be nested. In 5.13.2, MathStructure::integrate() has a path that returns without closing a hold when it solves a definite integral with an infinite bound. With one hold left open, all later warnings and errors go into it and never reach the user. The official qalc behaves the same: after integrate(e^(−x^2), −∞, ∞), calculating 1/0 does not give “Division by zero”.
- The fix: between commands, and when messages are collected, no hold should be open. So the bridge program closes all open holds at the following two points (it calls endTemporaryStopMessages until that reports that no hold is open).
- ① Before collecting messages (collect_messages): their contents move to the message list. They belong to the calculation that has just ended, so they are returned as its messages.

- ② Before running a command (the loop in main): their contents are dropped (they are left over from the previous command and must not be mixed with the messages of the next one).
- If a future libqalculate version fixes this, no hold is open, and these steps do nothing. Tests M03 to M05 check this (Test specification and results).

libqalculate behaviors to know about for calculus and matrices

Behavior	How the app handles it
limit() always gives the warning “limit() is incomplete and unreliable.”	The warning is shown (in Japanese on the Japanese screen), and the result still comes
dsolve() solves first-order differential equations only	The key template is dsolve(diff(y, x) =), and the message for equations it cannot solve is translated on the Japanese screen
Infinite series sum(…, 1, ∞) are not calculated	The key template is sum(, 1, 10). The examples have finite sums and products only
One-letter names (a, b, m, t and others) are read as units	The Calculus keys and the examples use x, y, z and n as variables
Matrices can be typed as `[1, 2; 3, 4]`, `[1 2; 3 4]` or `[[1, 2], [3, 4]]`, and are shown as one line of text, `[1 2; 3 4]`	The Matrix keys type [ ] and ;, and the page shows results as a table (3.9)

3.20 QCalc Pro (1.3.0)

QCalc Pro is a one-time in-app product on Google Play (product ID qcalc_pro). It unlocks calculations with more than 1,000 significant digits, 3D graphs, color themes, exporting the history (CSV) and the Pro features of later versions. Everything else stays free. Java (Pro and PlayBilling) keeps the state of the purchase, and the page (app.js) checks it before using a Pro feature.

Parts and their roles

Part	Role
Pro (Pro.java)	Keeps the state (bought: owned; payment pending: pending; earlier user: legacy; the price) and saves it in the SharedPreferences “pro”. Decides the kind of billing, gives it to the page with info(), and sends a pro event when the state changes
PlayBilling (app/src-billing)	Connects to Google Play with Google Play Billing Library 8, asks for the price and the purchases, opens the purchase screen and acknowledges purchases. It implements Pro.Store and reports results to Pro through Pro.Owner. It is only in builds made with Gradle and qcalc.billing=true; Pro loads the class by name (and works in builds without it)
The page (app.js)	Reads the JSON of proInfo() into PRO (readPro) and checks hasPro() before using a Pro feature. Without Pro it shows a prompt (proPrompt). The Pro panel (openPro) takes purchases and restores

Kinds of billing

billing	Build	Pro features	The screen
play	A build with billing (PlayBilling included)	Usable after a purchase, or for an earlier user	“QCalc Pro” in the menu, and a PRO mark on the Pro features while they are locked
debug	The test build (package name ending in .debug)	Usable from the start. “Pro unlocked (for testing)” in the Pro panel can lock them (setTestPro; saved as test_locked)	As play. When locked, it looks as it does without Pro
none	A build without billing (made by build-apk.sh or build-aab.sh)	All usable, as Pro cannot be bought	No “QCalc Pro” item, and nothing is restricted

hasPro() is true when billing is neither play nor debug, or when pro is true. The page reads proInfo() again at each pro event (proChanged).

Using a Pro feature (the page)

Feature	Without Pro
Calculating with more than 1,000 significant digits	The settings field allows up to 1,000; a larger number is set back to 1,000 and the prompt is shown. The precision sent to the engine is also kept at 1,000 or less (engineOpts)
3D graphs	The 3D button shows the prompt and keeps the 2D graph. A saved kind of 3d also opens as 2D
Color themes	Choosing a color other than Blue sets it back to Blue and shows the prompt. A saved theme is shown as Blue (applyTheme)
Exporting the history	“Export history (CSV)” in the menu shows the prompt

The prompt (proPrompt) is a sheet saying “… is a QCalc Pro feature”, whose “About QCalc Pro” button opens the Pro panel. While Pro is locked, the html element has the pro-locked class, and the Pro items (elements with a data-pro attribute) show a “PRO” mark. When Pro becomes available, “QCalc Pro is unlocked. Thank you!” is shown, the color theme and the precision limit are applied, and the expression being typed is calculated again.

Buying and checking (PlayBilling)

1. At the very start (MainActivity.onCreate) Pro is made, and start connects to Google Play and asks for the price and the purchases. It asks again each time the app comes back to the screen (onResume) and when the Pro panel opens, to find purchases made outside the app (such as a promo code redeemed in the Play Store) and payments completed later, and to stop counting refunded purchases.
2. “Buy QCalc Pro”: if the price information (ProductDetails) is not there yet, it is asked for first; then the purchase screen opens (launchBillingFlow). The purchase option (offer token) is the backward-compatible one if there is one, otherwise the first.
3. handle checks the results of the purchase screen and of the queries. When a purchase of qcalc_pro is PURCHASED, owned is set, and the purchase is acknowledged if it is not yet (Google Play refunds purchases not acknowledged within 3 days; on a failure, it is tried again at the next query). When it is PENDING, the state becomes pending and “The payment is pending. QCalc Pro will be unlocked when it is complete.” is reported.
4. “Restore a purchase”: the purchases are asked for again, and when none is found, “No purchase of QCalc Pro was found” is reported.

5. Pro saves the state in the SharedPreferences “pro” and sends a pro event (`{"event":"pro"}`, with message when there is something to report) to the page.
6. When connecting or asking fails, the state stays as it was (a bought Pro stays usable). Failures are reported only when the user did something (buying or restoring).

Google Play handles the purchase screen and the payment; the app receives no payment information. PlayBilling's texts are in English, and on the Japanese screen the page shows them in Japanese (3.14).

Earlier users (legacy)

People who used QCalc before Pro began can use the Pro features without buying it (LEGACY_USERS_KEEP_PRO = true).

- Each start of a build without billing sets the mark pre_pro.
- At the first start of a build with billing (when the mark era is not there yet), legacy is set if pre_pro is there, if files/state holds saved state, or if the WebView's data folder (app_webview) exists. This is checked before the WebView or the engine writes anything (at the very start of onCreate).
- So people who used a build without billing (including the 1.3.0 App Bundle) can use the Pro features for free even after updating to a build with billing.
- legacy is a mark on the device, so it does not come back after the app's data is cleared. A purchase (owned) is recorded with the Google account, so it comes back through the query after installing again.
- For legacy, the Pro panel shows “It is free for you, because you used QCalc before QCalc Pro began. Thank you!”

Releases and billing

build-apk.sh and build-aab.sh do not include PlayBilling, so the versions they make (including the 1.3.0 App Bundle) have billing none, and all the Pro features can be used. A version in which Pro can be bought is built with Gradle and qcalc.billing=true (which gets the Google Play Billing Library from Google's Maven repository; Build and maintenance manual).

3.21 Language of the screen (1.3.0)

The screen is in English or Japanese, and English is the default. It is in Japanese only when the user chose Japanese with “Language / 言語” in the menu or the settings (the phone's language setting is not followed).

- Saving the language: setLanguage saves the chosen language as qalc.lang (files/state/qalc.lang.json; "en" or "ja"). lang in the settings (qalc.settings) only mirrors the language in use and is set to the value of qalc.lang at start. The language is saved

apart from the settings because the test builds of 1.3.0 saved it with the other settings (so that the app never starts in a language the user did not choose).

- At start: app.js first reads qalc.lang to decide LANG (en or ja) and JA, and sets the lang attribute of the html element. The language does not change while the page runs.
- Choosing texts: the page's texts are written side by side as tr(English, Japanese). The fixed texts of index.html are in English and are replaced at start only for Japanese (localizeStatic).
- Changing the language: setLanguage saves qalc.lang and the settings, saves the history and the expression being typed at once (qalcPause), and reloads the page. The history, the expression being typed, ans and user definitions stay. Resetting the settings to their defaults does not change the language.
- Texts of the engine and Java: the texts that the engine (libqalculate and qalc-engine.cc) and Java return to the page are in English. On the Japanese screen, the page turns them into Japanese with MESSAGES_JA (57 entries; localMessage). Texts not in the table are shown in English (3.14).
- Texts that Java shows itself (when the WebView cannot be used, when the screen stops again and again, the notice after copying): MainActivity.tr(English, Japanese) chooses by reading qalc.lang.
- What changes with the language: the currency expressions on the first screen and in the examples (100 USD to EUR and others in English; conversions to yen in Japanese), the buttons at the top (DEC, HEX and others, or 10進, 16進 and others), the spoken names of the keys, the CSV headings, and where the privacy policy opens (at its Japanese part for Japanese). The Japanese names (of functions, variables and units), searching in Japanese and the descriptions of the Japanese units are only on the Japanese screen (3.18).
- What does not change: the calculations (the local currency depends on the phone's region: JPY in Japan), how expressions are written (function and unit names stay in English), and the conversion of full-width characters and the like (normalizeJa).

3.22 Color themes (1.3.0)

The color of the keys and the highlights can be chosen from 7 (the setting “Color theme”, theme). This is a QCalc Pro feature (3.20).

Theme (theme)	Accent color (light)	Accent color (dark)
Blue (blue; the default)	#2563EB	#6B9CFF
Green (green)	#15803D	#4ADE80

Theme (theme)	Accent color (light)	Accent color (dark)
Teal (teal)	#0F766E	#2DD4BF
Purple (purple)	#7C3AED	#A78BFA
Rose (rose)	#BE123C	#FB7185
Orange (orange)	#C2410C	#FB923C
Graphite (graphite)	#374151	#CBD5E1

- The colors are CSS variables in style.css: `--accent` (the accent color), `--accent-text` (text on the accent color), `--accent-soft` (a light accent), `--key-op` (the background of the operator keys) and `--key-op-text` (the text of the operator keys). Blue is the default of style.css; the other 6 override these through the data-theme attribute of the html element, with values for both light (`:root[data-theme=…]`) and dark (`:root.dark[data-theme=…]`).
- applyTheme sets the dark class by dark mode (dark from appInfo(), or prefers-color-scheme when it is missing), and sets data-theme only when Pro is usable and the theme is not Blue. It is called at start, when the settings change and when the Pro state changes.
- The colors of the system bars (3.15) do not change with the theme.

3.23 Exporting the history (CSV; 1.3.0)

“Export history (CSV)” in the menu saves the history as a CSV file. This is a QCalc Pro feature (3.20).

1. exportHistory: without Pro it shows the prompt; with an empty history it shows “The history is empty” and stops.
2. historyCsv makes the CSV: UTF-8 with a byte order mark at the start (so that Excel reads it as UTF-8), and CRLF line ends. The first line has the headings (Date, Expression, Result and Number; 日時, 式, 結果 and 数値 on the Japanese screen).
3. Each row has the date and time (YYYY-MM-DD HH:MM), the expression, the result as shown (without a leading “= ”; a long result as “start … end”) and the number (only for results that a single decimal number can show, with the group spaces removed and the minus as -; empty for comparisons, cut results and results not in base 10). Every field is enclosed in `"`, with each `"` inside doubled to `""`. When the expression or the result starts with =, +, -, @, a tab or CR, a `'` is put before it (so that spreadsheets do not run it as a formula).

4. The file name is QCalc-history-YYYYMMDD.csv (for example, QCalc-history-20261113.csv).
5. In the app, QalcNative.saveFile(name, "text/csv", contents) is called. MainActivity checks the name, the type and the length (3.2) and opens the system file picker (ACTION_CREATE_DOCUMENT). The contents are kept in memory while the user chooses (pendingSave).
6. When a place is chosen (onActivityResult), the qalc-save thread opens it with "wt" (replacing its contents), writes in UTF-8 and reports the result with a saved event (ok on success, cancelled when the picker was closed without choosing, and error “Could not save the file” on a failure). If the app was re-created while the user chose and the contents are gone, that is an error too.
7. The page (fileSaved) shows “History exported” on success and “Could not save the file” on a failure. Nothing is shown when the picker was closed without choosing.
8. When checking in a PC browser (without QalcNative), the same contents are downloaded as a Blob.

The app does not remember where it saved the file, and it never reads the file again or sends it anywhere. The file is outside the app's private storage, so it stays when the app is removed.

Example (English screen; a file exported in a test)

```
"Date","Expression","Result","Number"
"2026-11-13 02:53","1/3","≈ 0.3333333333","0.3333333333"
"2026-11-13 02:53","−5 + 2","−3","-3"
"2026-11-13 02:53","'=1+1","false",""
"2026-11-13 02:53","2 m + 30 cm","2.3 m",""
```